

Các công nghệ lập trình hiện đại

# **Thiết kế mẫu (Design Pattern) trong Lập trình hướng đối tượng**

TS. Đỗ Như Tài  
Đại Học Sài Gòn  
[dntai@sgu.edu.vn](mailto:dntai@sgu.edu.vn)

# Mục Lục

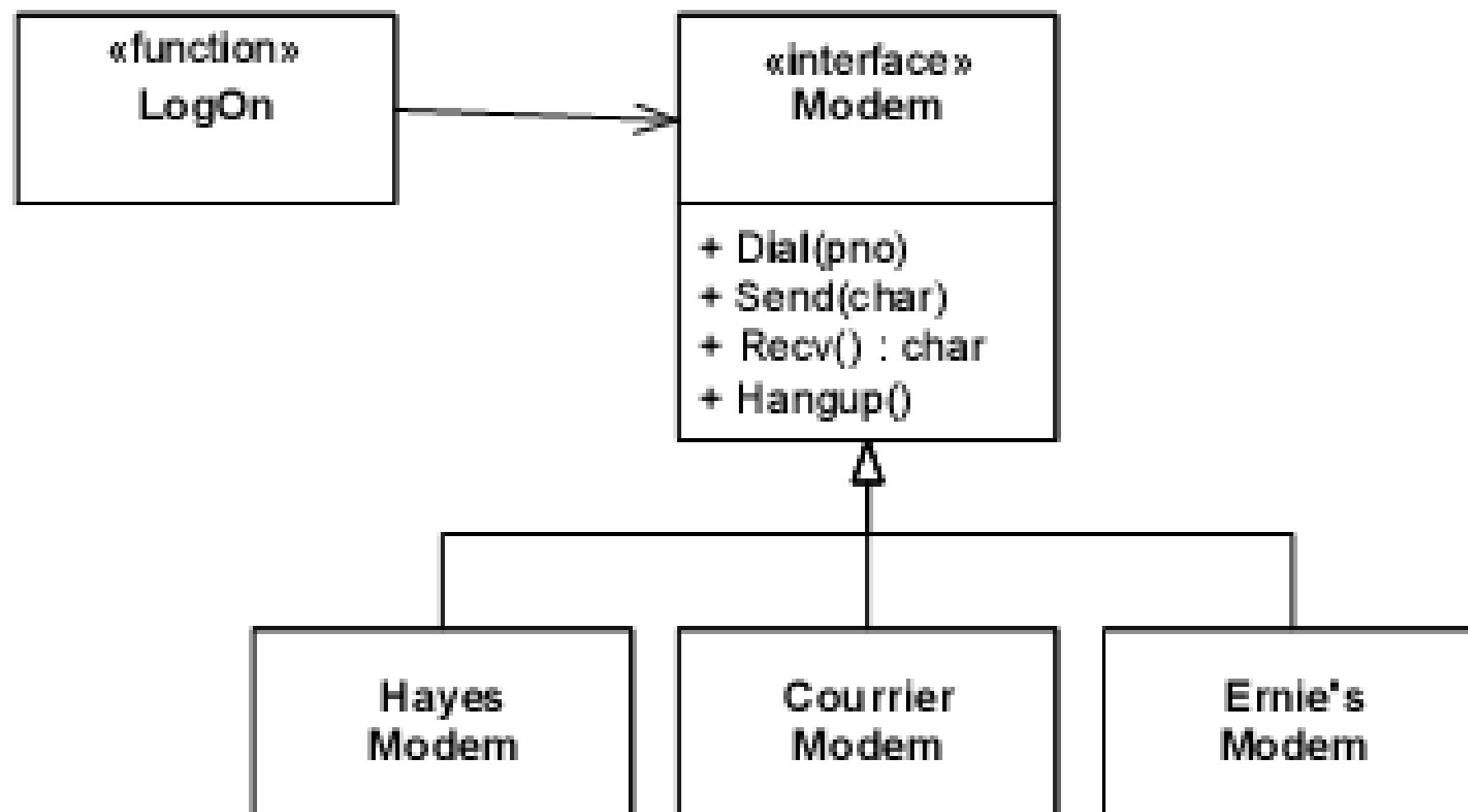
- Các nguyên lý thiết kế hướng đối tượng
- Thiết kế mẫu (Design Patterns)
- 23 mẫu của GoF
- Ví dụ minh họa
- Tài liệu tham khảo

# Các nguyên lý thiết kế hướng đối tượng

# Nguyên lý đóng mở

## Nguyên lý đóng mở (**Open-Closed Principle**)

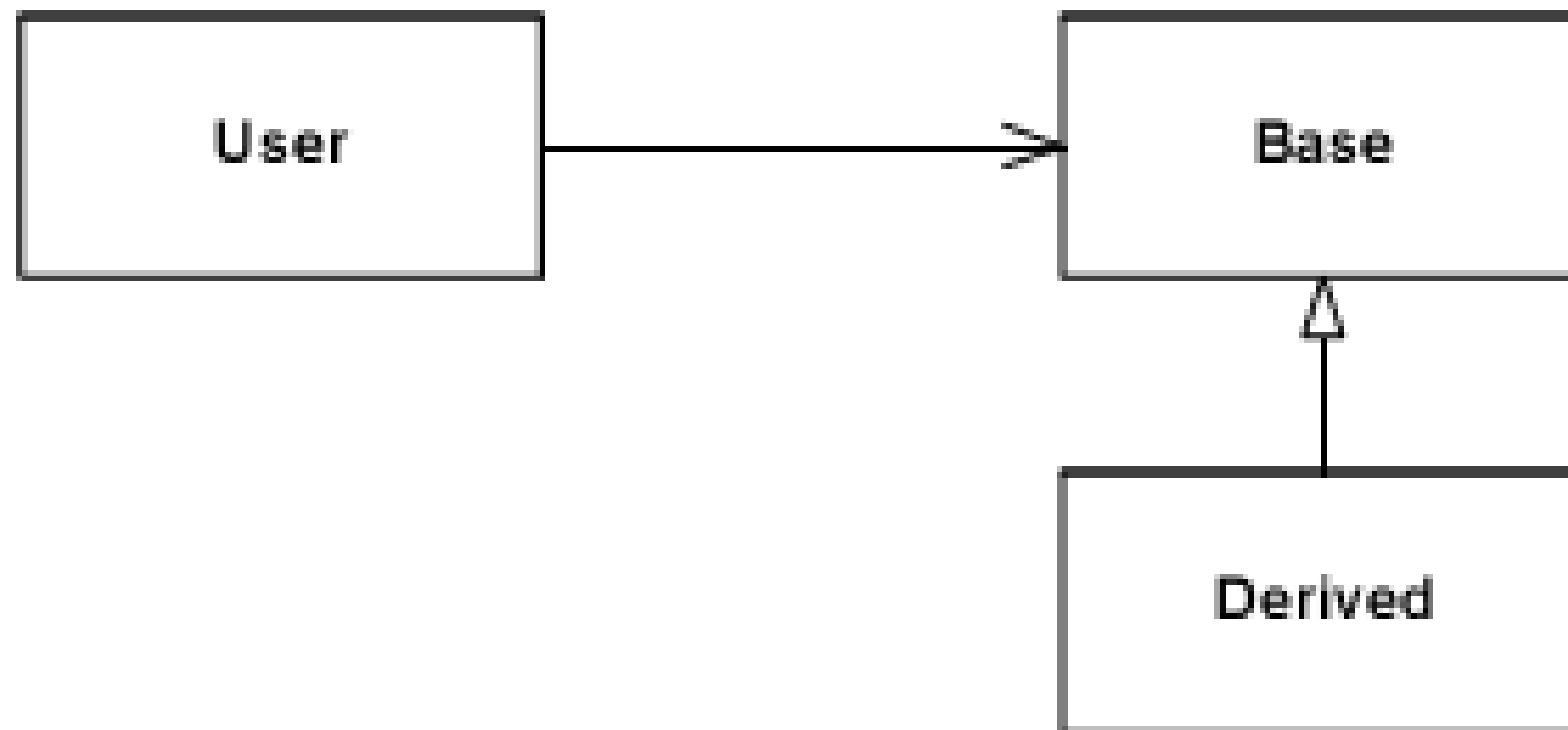
- ✓ Một module cần “mở” đối với việc phát triển thêm tính năng nhưng phải “đóng” (hạn chế hoặc không cho phép) đối với việc sửa đổi mã nguồn.



# Nguyên lý thay thế Liskov

Nguyên lý thay thế Liskov (**Liskov Substitution Principle**)

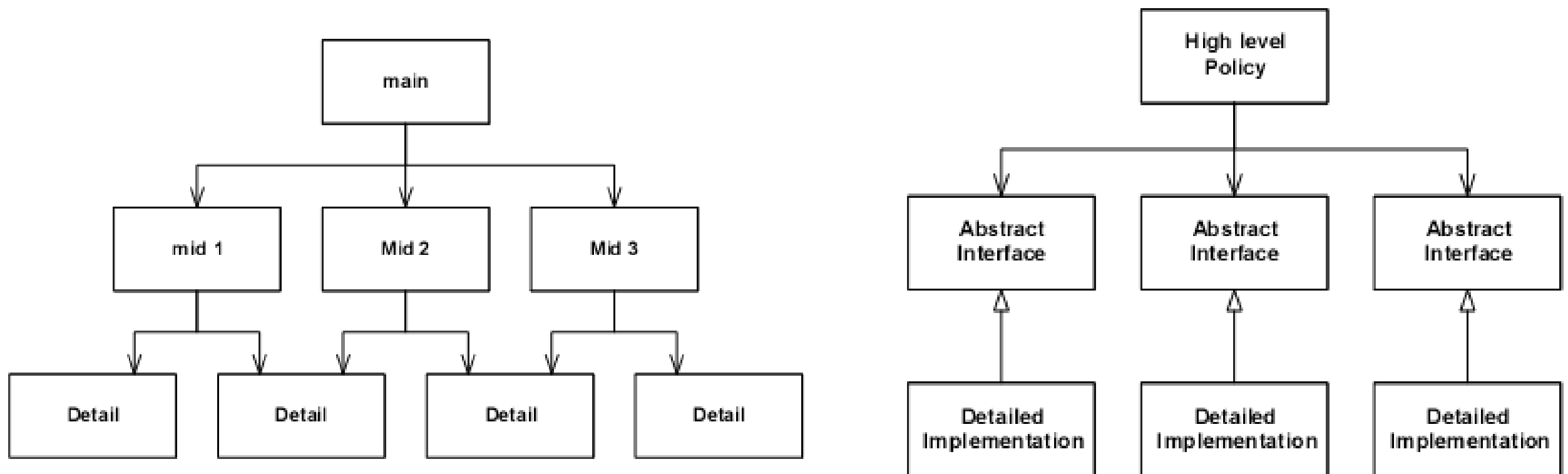
- Các lớp cơ sở đều có thể thay thế bằng các lớp con (lớp kế thừa).
- Ý nghĩa là các chức năng của chương trình vẫn thực hiện đúng nếu ta thay bất kì một lớp đối tượng nào đó bằng đối tượng kế thừa.



# Nguyên lý nghịch đảo phụ thuộc

Nguyên lý nghịch đảo phụ thuộc (**Dependency Inversion Principle**)

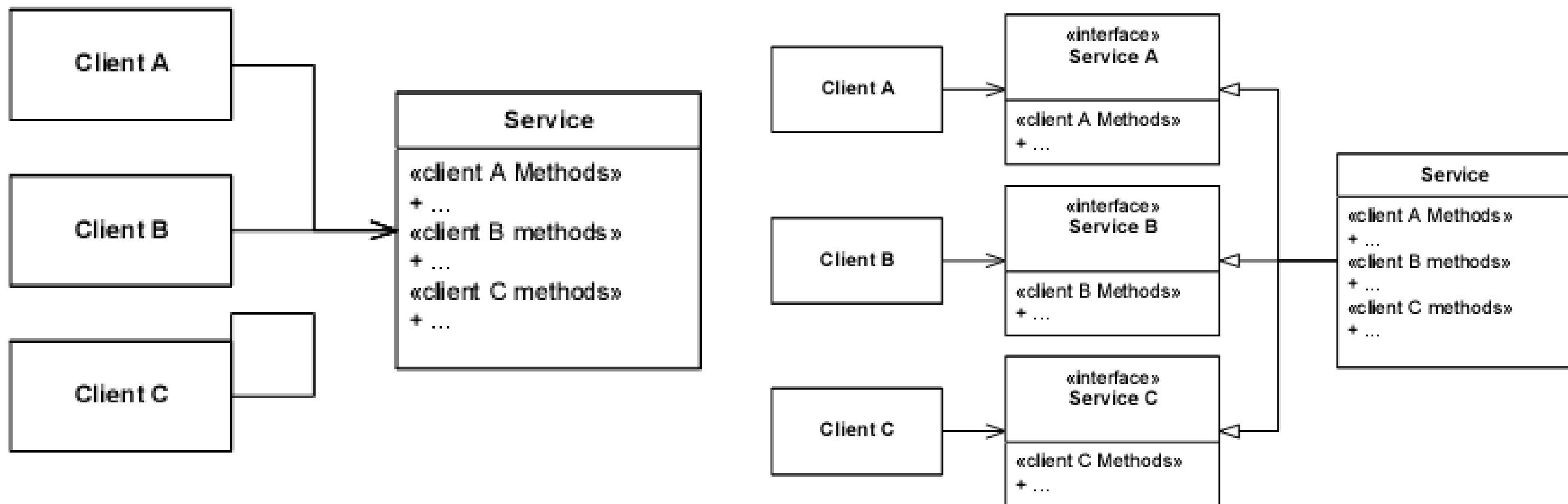
- Phụ thuộc vào mức trừu tượng, không phụ thuộc vào mức chi tiết.



# Nguyên lý phân tách giao diện

## Nguyên lý phân tách giao diện (**Interface Segregation Principle**)

- Nên có nhiều giao diện đặc thù với bên ngoài hơn là chỉ có một giao diện dùng chung cho một mục đích.





# Thiết kế mẫu (Design Pattern)



# Thiết kế mẫu (Design Patterns - DP)

- Design Pattern là bài toán thông dụng cần giải quyết và cách giải quyết bài toán đó trong một ngữ cảnh cụ thể.
- DP đóng vai trò là sáng kiến để giải quyết một bài toán thông dụng nào đó. Từ đó giúp xác định bài toán giải gặp phải nhanh gọn và hợp lý.
- Tuân thủ nghiêm ngặt các nguyên lý thiết kế hướng đối tượng, từ đó giúp hệ thống có khả năng tái sử dụng cao.

## Các thành phần của mẫu

- **Name** (tên mẫu): giúp hình dung nhanh chóng về mẫu mà không cần phải xem lại cấu trúc mẫu.
- **Intent** (mục đích): mô tả vắn tắt và mục đích sử dụng.
- **Also Known As**: các tên gọi khác.
- **Motivation**: ngữ cảnh mà mẫu sử dụng.
- **Applicability**: các tình huống áp dụng mẫu.
- **Structure** : cấu trúc của mẫu thể hiện dưới dạng sơ đồ.
- **Participants** (thành phần): mô tả các thành phần tham gia vào mẫu.

## Các thành phần của mẫu

- **Collaborations** : tương tác giữa các thành phần.
- **Consequences** (hệ quả): kết quả ứng dụng mẫu, ưu điểm, nhược điểm.
- **Implementation** (cài đặt)
- **Sample Code**: chương trình mẫu.
- **Known Uses**: ứng dụng thực tế được biết đến.
- **Related Patterns**: các mẫu liên quan.

Trong đó, bốn thành phần cơ bản xác định pattern: **name**, **problem**, **solution**, **consequences**.

# Phân loại mẫu

Có hai cách phân loại các Pattern của GoF :

- Phân loại theo mục đích sử dụng:
  - **Creational** (mẫu kiến tạo): trừu tượng hóa quá trình khởi tạo đối tượng.
  - **Behavioral** (mẫu tương tác): mô tả sự tương tác giữa các đối tượng và cách chúng phân phối, cộng tác để giải quyết một hay một nhóm trách nhiệm nào đó.
  - **Structural** (mẫu cấu trúc): cách các đối tượng và lớp đối tượng kết hợp để tạo nên cấu trúc lớn hơn và hữu dụng hơn.
- Phân loại theo phạm vi:
  - **Class**: mô tả và giải quyết mối quan hệ giữa các lớp đối tượng và lớp con của chúng. Các mối quan hệ này thành lập dựa trên cơ chế kế thừa nên xảy ra tại thời điểm biên dịch (cố định).
  - **Object**: mô tả và giải quyết mối quan hệ giữa các đối tượng. Các mối quan hệ này có thể thay đổi tại thời điểm run-time.

# 23 mẫu của GoF

# Bảng tuần hoàn GoF

## The Sacred Elements of the Faith

|                  |                               |                              |                      |                       |                       |                                      |                        |                       |
|------------------|-------------------------------|------------------------------|----------------------|-----------------------|-----------------------|--------------------------------------|------------------------|-----------------------|
| the holy origins | <b>FM</b><br>Factory Method   |                              |                      |                       |                       |                                      | <b>A</b><br>Adapter    | the holy structures   |
|                  | <b>PT</b><br>Prototype        | <b>S</b><br>Singleton        | the holy behaviors   |                       |                       | <b>CR</b><br>Chain of Responsibility | <b>CP</b><br>Composite | <b>D</b><br>Decorator |
|                  | <b>AF</b><br>Abstract Factory | <b>TM</b><br>Template Method | <b>CD</b><br>Command | <b>MD</b><br>Mediator | <b>O</b><br>Observer  | <b>IN</b><br>Interpreter             | <b>PX</b><br>Proxy     | <b>FA</b><br>Façade   |
|                  | <b>BU</b><br>Builder          | <b>SR</b><br>Strategy        | <b>MM</b><br>Memento | <b>ST</b><br>State    | <b>IT</b><br>Iterator | <b>V</b><br>Visitor                  | <b>FL</b><br>Flyweight | <b>BR</b><br>Bridge   |

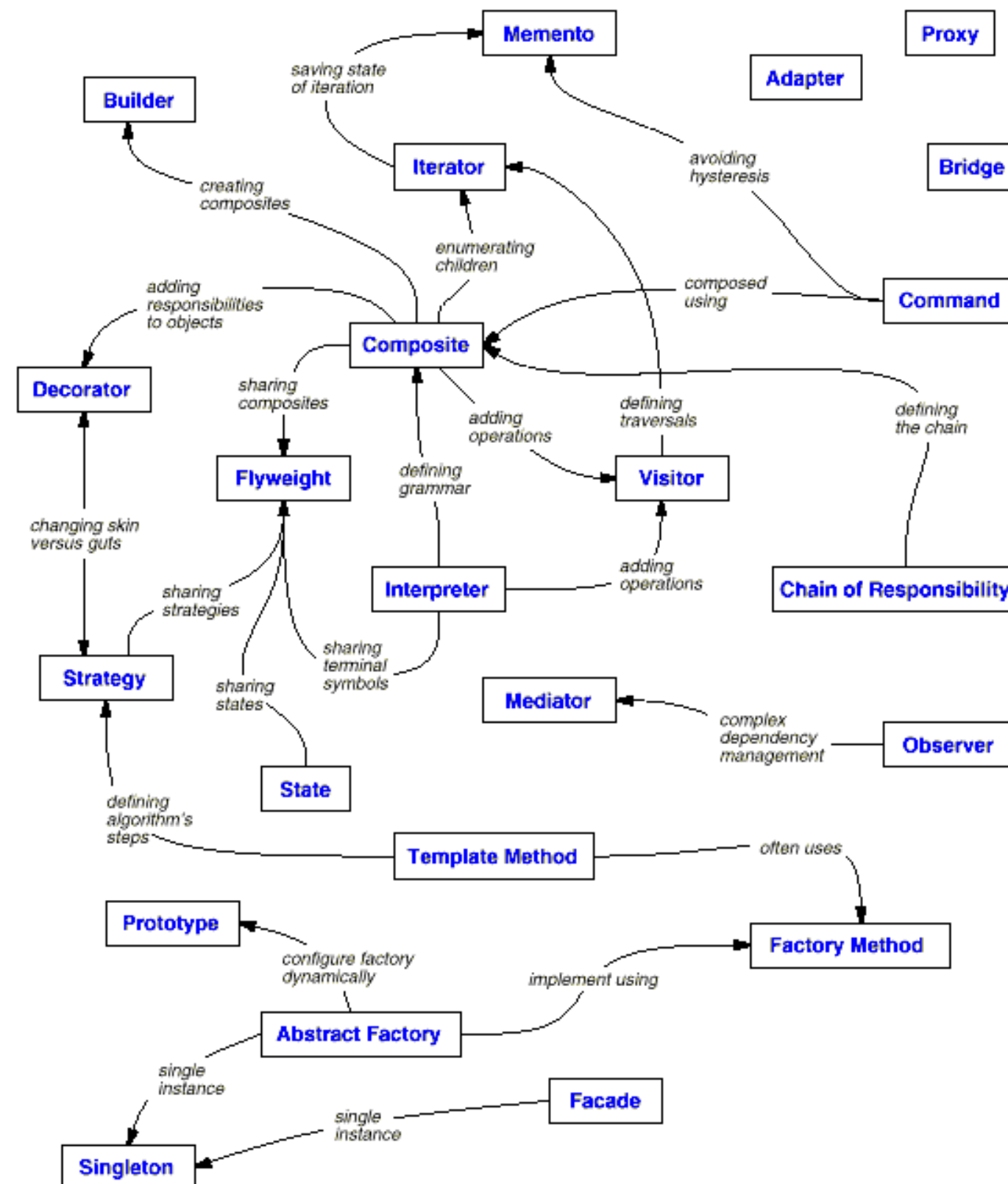
**Colorized GoF periodic table**

# Bảng phân loại mẫu

|         |        | Mục đích sử dụng                                      |                                                                |                                                                                                                                |
|---------|--------|-------------------------------------------------------|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
|         |        | Creational                                            | Structural                                                     | Behavioral                                                                                                                     |
| Phạm vi | Class  | Factory Method                                        | Adapter                                                        | Interpreter<br>Template Method                                                                                                 |
|         | Object | Abstract Factory<br>Builder<br>Prototype<br>Singleton | Adapter<br>Bridge<br>Composite<br>Decorator<br>Facade<br>Proxy | Chain of Responsibility<br>Command<br>Iterator<br>Mediator<br>Memento<br>Flyweight<br>Observer<br>State<br>Strategy<br>Visitor |



# Mối quan hệ giữa các mẫu



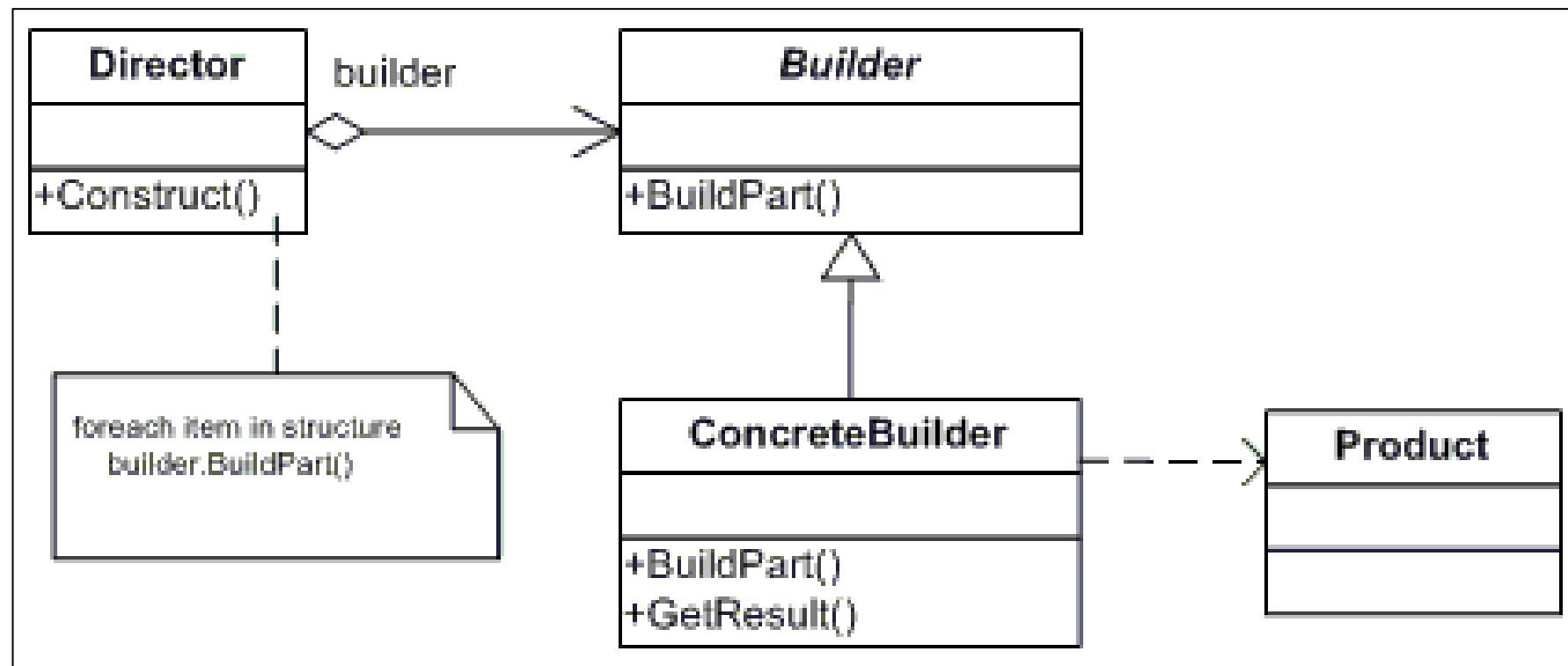


# Creational Pattern

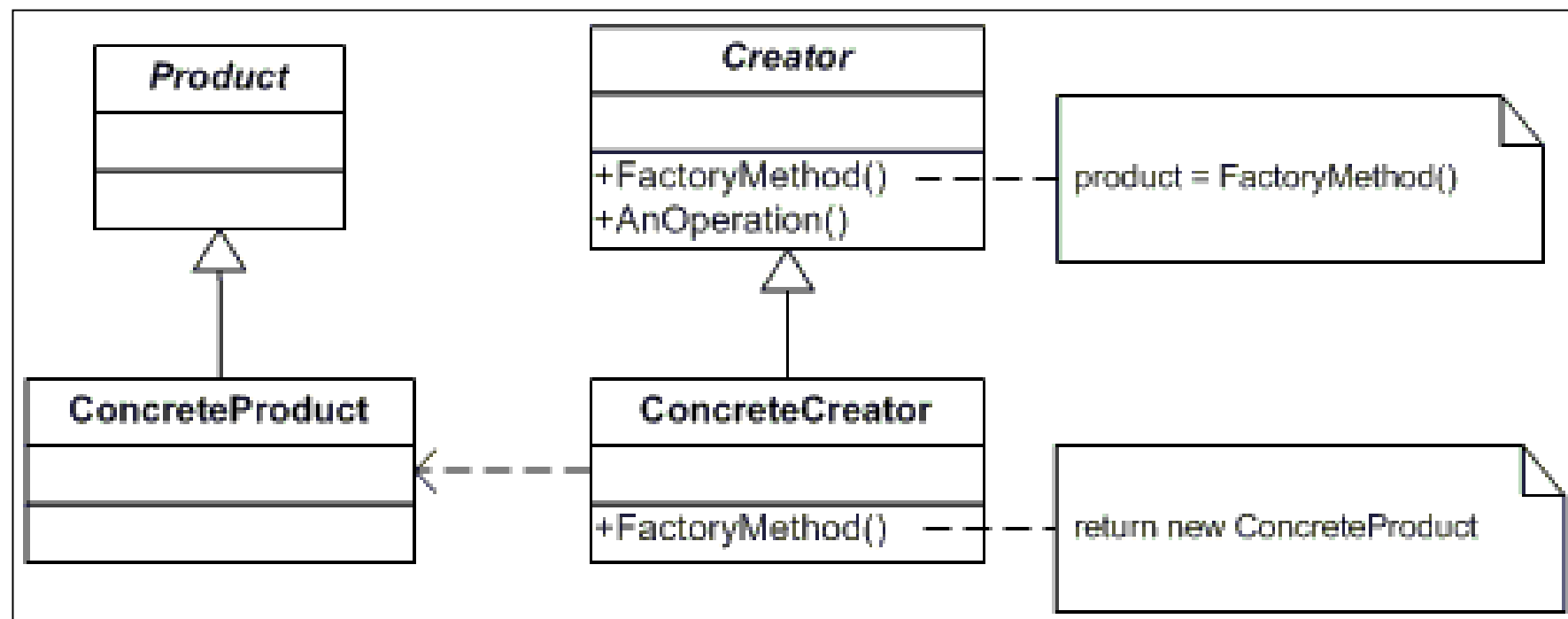
- **Abstract Factory:** cung cấp giao diện cho việc tạo một lớp các đối tượng quan hệ với nhau mà không cần quan tâm đến lớp cụ thể.
- **Builder:** phân tách những khởi dựng của một đối tượng phức hợp từ một định dạng, để cùng một khởi dựng, ta có thể tạo nên nhiều định dạng khác nhau.
- **Factory Method:** tạo một instance thông qua những lớp kế thừa.
- **Prototype:** sao chép hay nhân bản một instance đã khởi tạo.
- **Singleton:** một lớp chỉ tồn tại duy nhất một instance.

# Creational Pattern

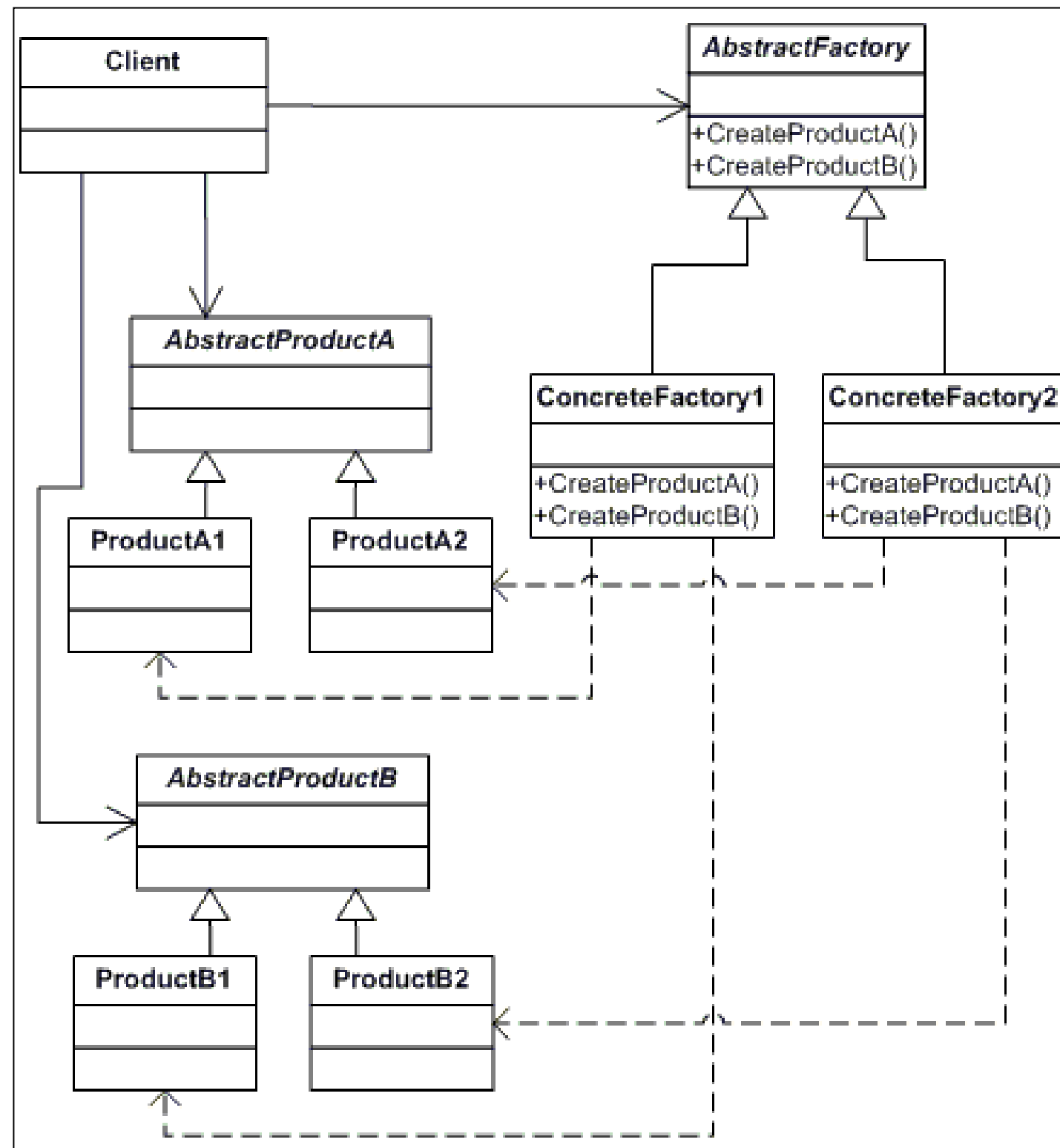
## Builder Pattern



## Factory Method Pattern



# Creational Pattern



Abstract Factory Pattern

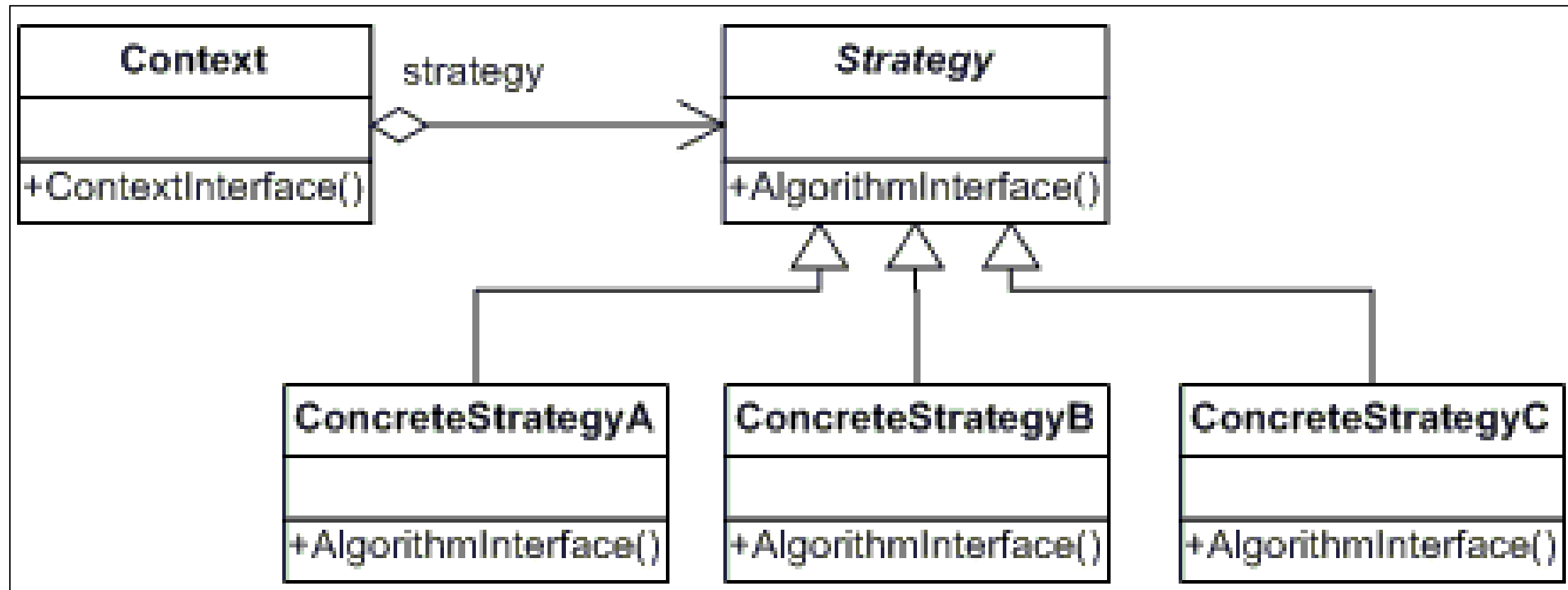
# Behavioral Pattern

- **Chain of Resp.:** một cách để chuyển yêu cầu giữa một chuỗi các đối tượng.
- **Command(\*):** gói một yêu cầu thành đối tượng, giúp tham số hóa những yêu cầu của client, và hỗ trợ cho thao tác undo.
- **Interpreter:** xây dựng kiến trúc ngữ pháp cho một ngôn ngữ.
- **Iterator(\*):** xử lý tuần tự các phần tử trong một collection.
- **Mediator:** định nghĩa một cách đơn giản sự liên lạc giữa các lớp.
- **Memento:** lưu giữ và phục hồi trạng thái bên trong của một đối tượng.

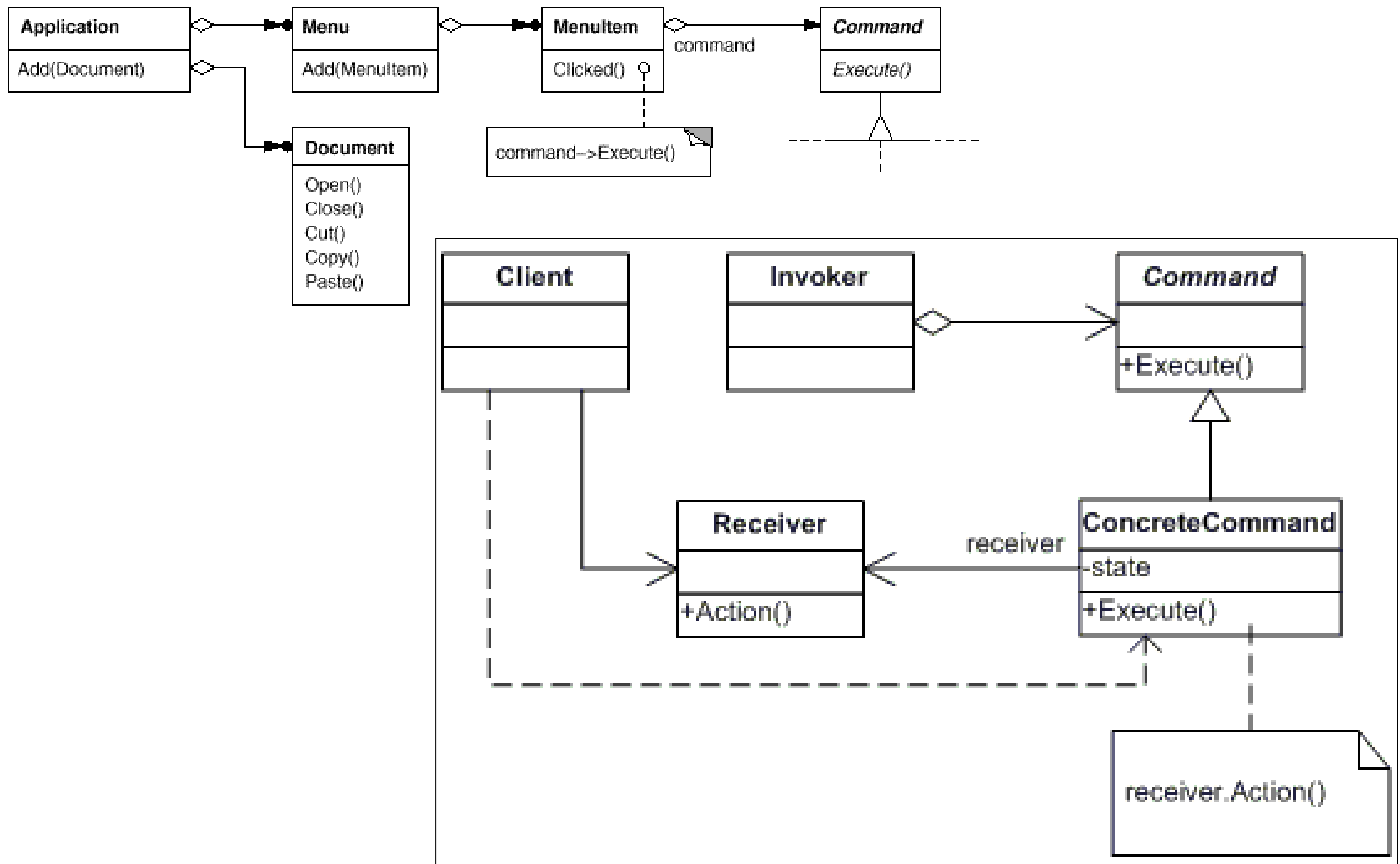
# Behavioral Pattern

- **Observer(\*)**: Định nghĩa mối quan hệ một nhiều giữa các đối tượng, để khi một đối tượng thay đổi trạng thái, tất cả đối tượng phụ thuộc được thông báo và cập nhật tự động.
- **State**: thay đổi hành vi đối tượng khi trạng thái của nó thay đổi.
- **Strategy(\*)**: định nghĩa một lớp thuật toán khác nhau, đóng gói và làm cho những thuật toán trong lớp có thể thay đổi cho nhau tạo nên sự độc lập khi client sử dụng.
- **Template Method**: định nghĩa khung sườn thuật toán của một hành động, và cho phép lớp con định nghĩa lại các hành động tùy thuộc vào ngữ cảnh cụ thể.
- **Visitor**: định nghĩa một thao tác mới đến lớp không tạo nên sự thay đổi ở lớp.

# Strategy pattern

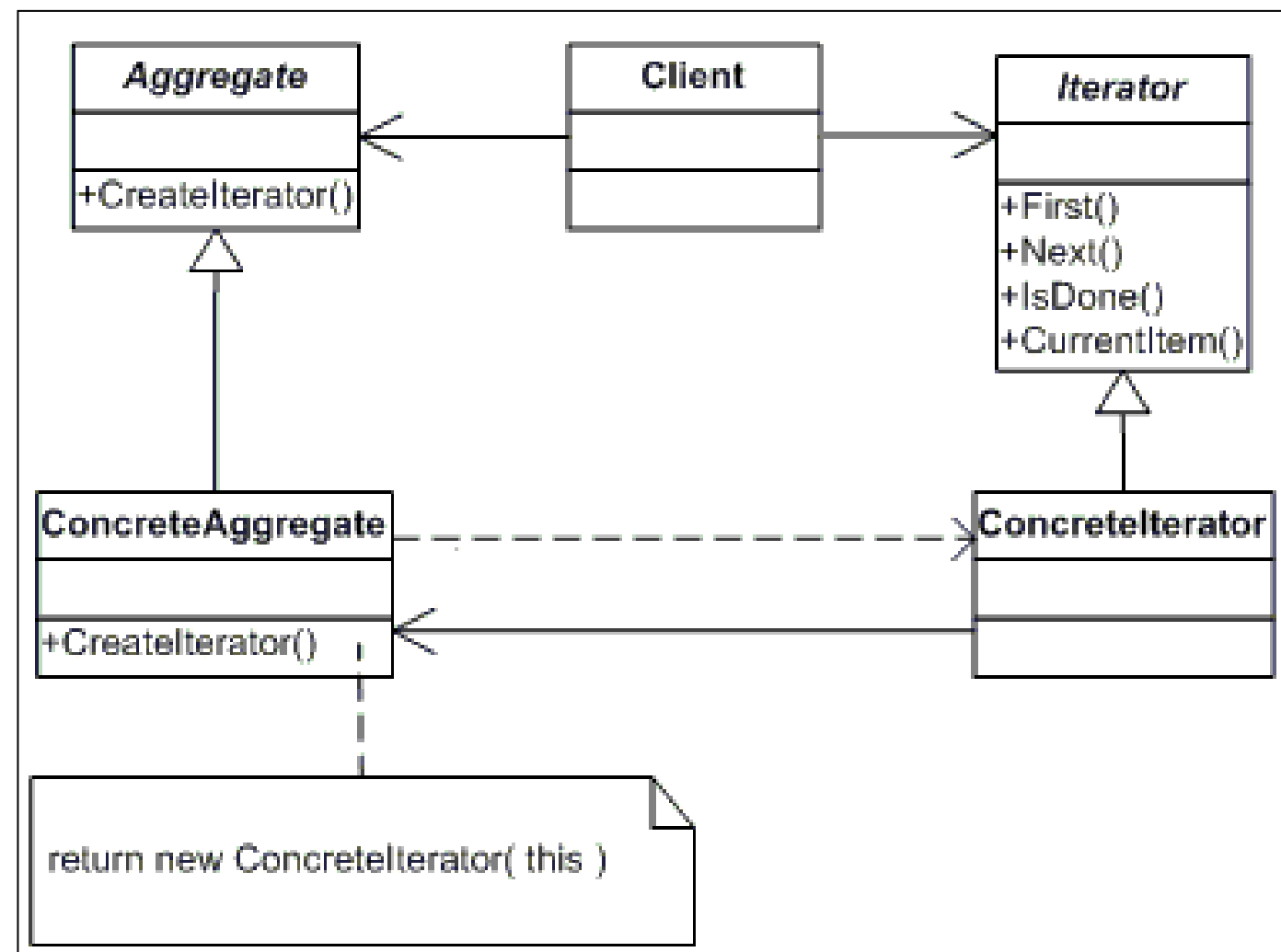
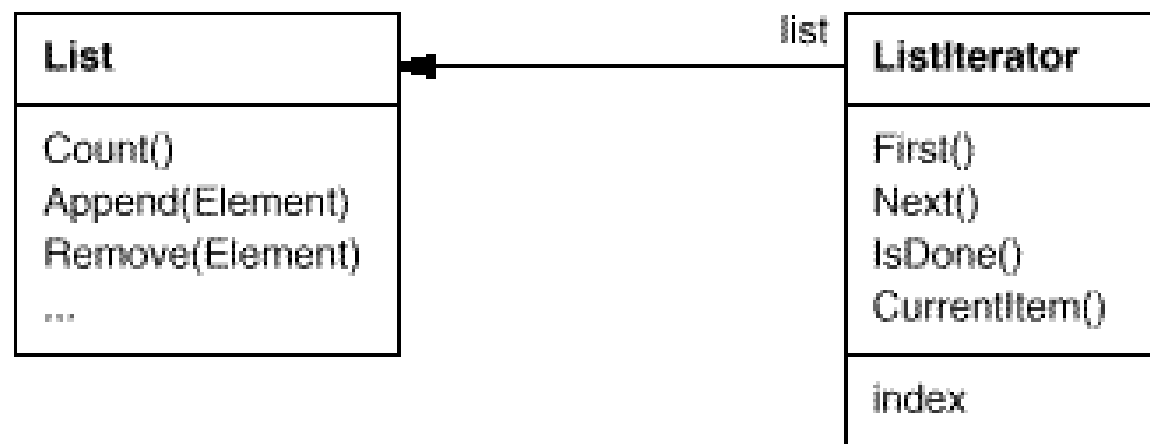


# Command pattern



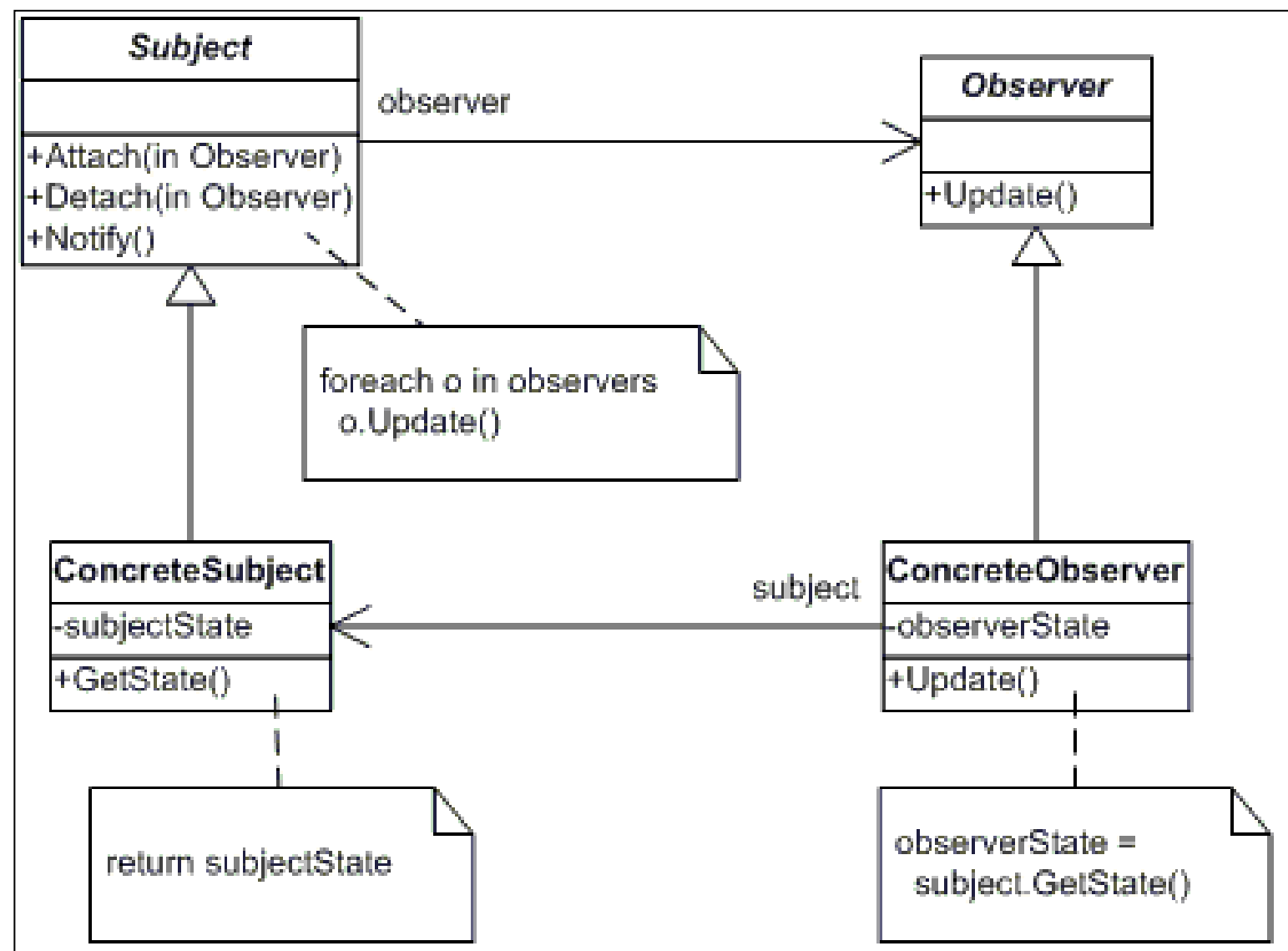
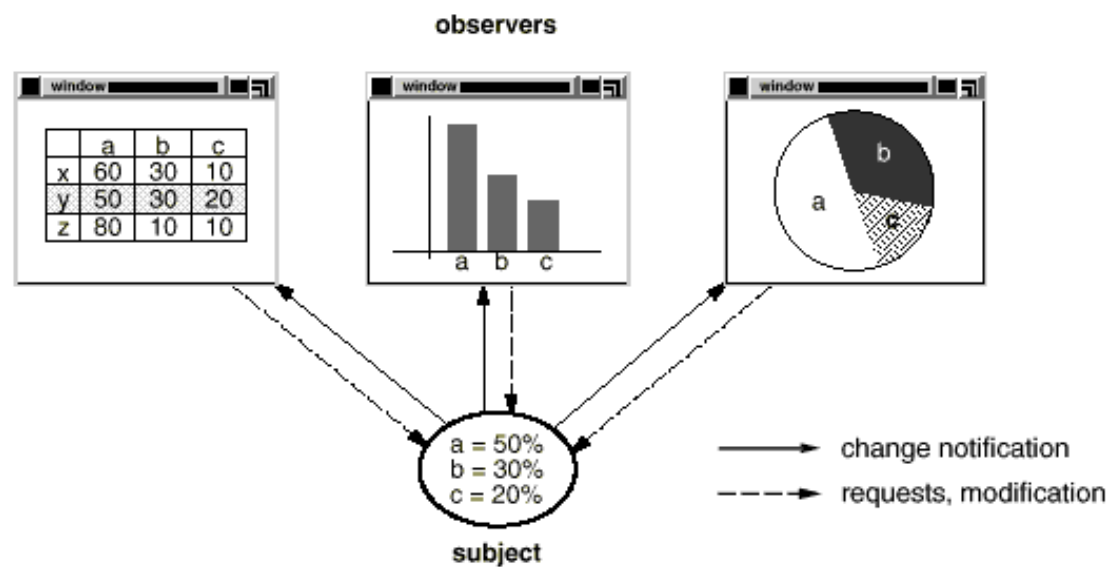


# Iterator pattern





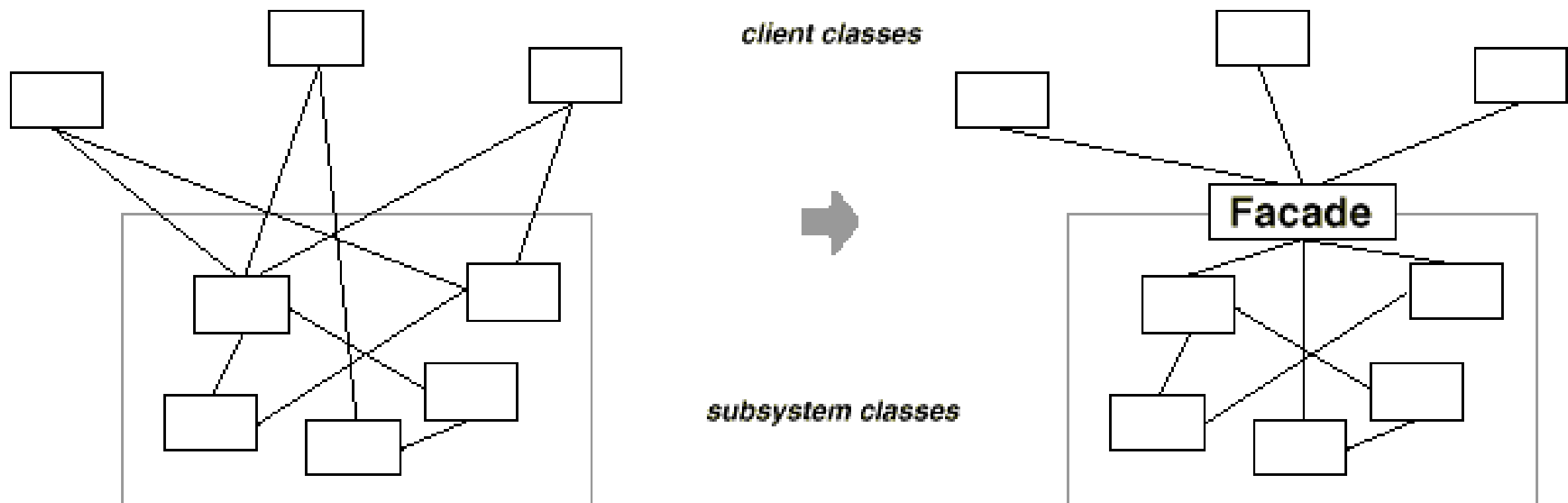
# Observer pattern



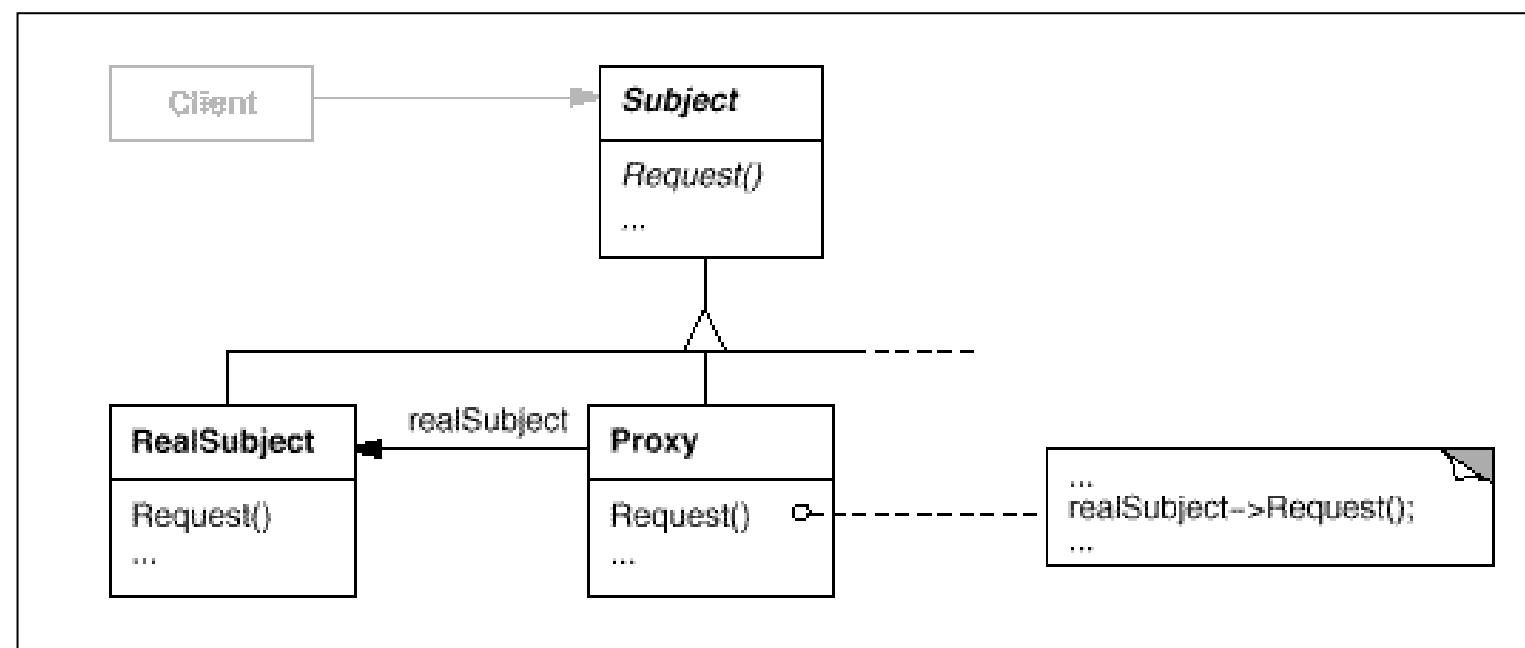
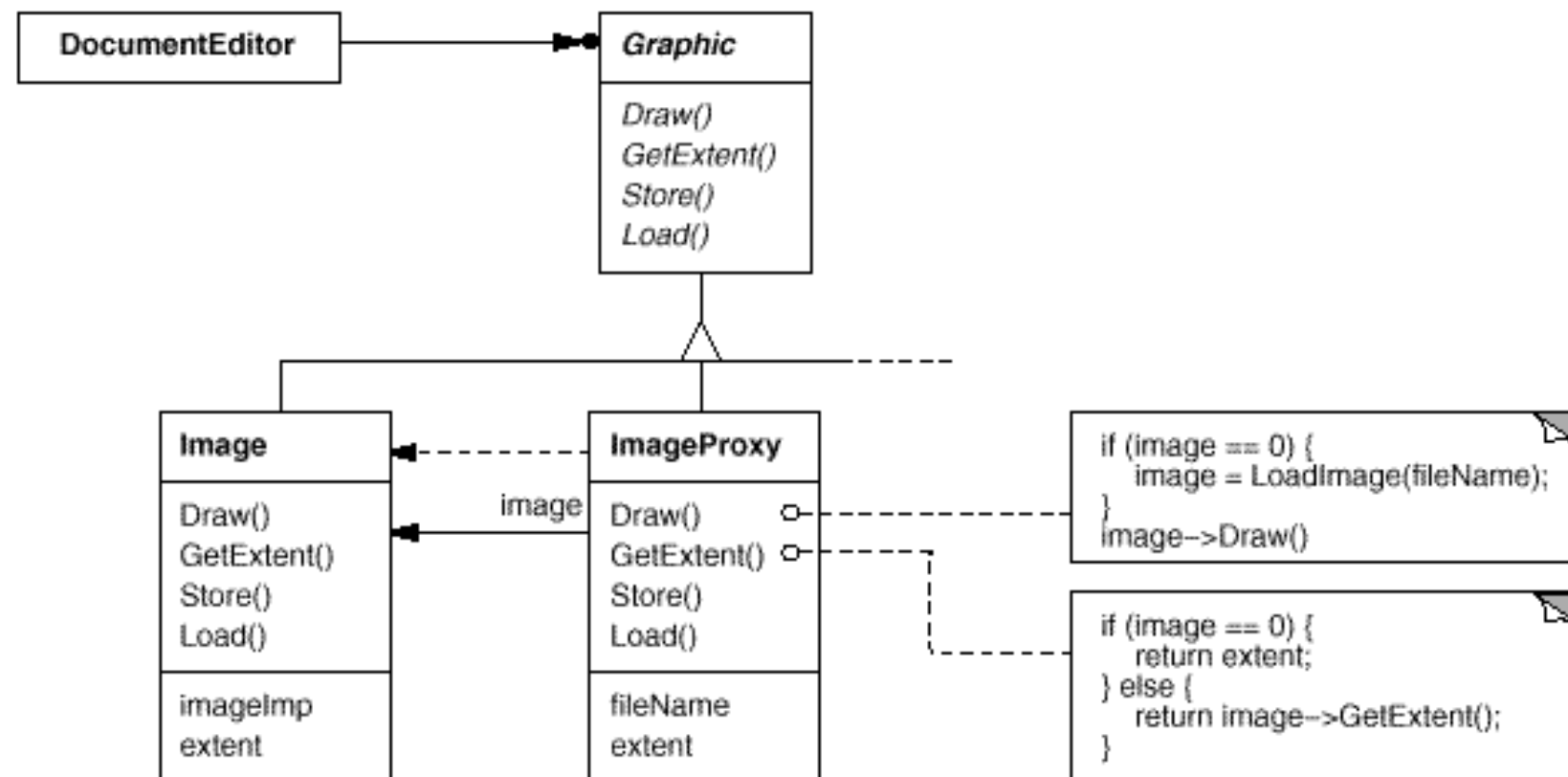
# Structural Patterns

- Adapter giao diện trung gian để gắn kết vào hệ thống một lớp đối tượng nào đó.
- Bridge tách thành phần ảo và thành phần thực thi riêng biệt.
- Composite gom các đối tượng vào trong cấu trúc hình cây để thể hiện được cấu trúc tổng quát của nó.
- Decorator bổ sung trách nhiệm cho đối tượng tại thời điểm chạy.
- Facade giao diện lập trình ở mức trừu tượng hơn những giao diện đã có làm cho hệ thống con dễ sử dụng hơn.
- Flyweight làm phương tiện dùng chung để quản lý một cách hiệu quả số lượng lớn các đối tượng nhỏ.
- Proxy đại diện một đối tượng phức tạp bằng đối tượng đơn giản.

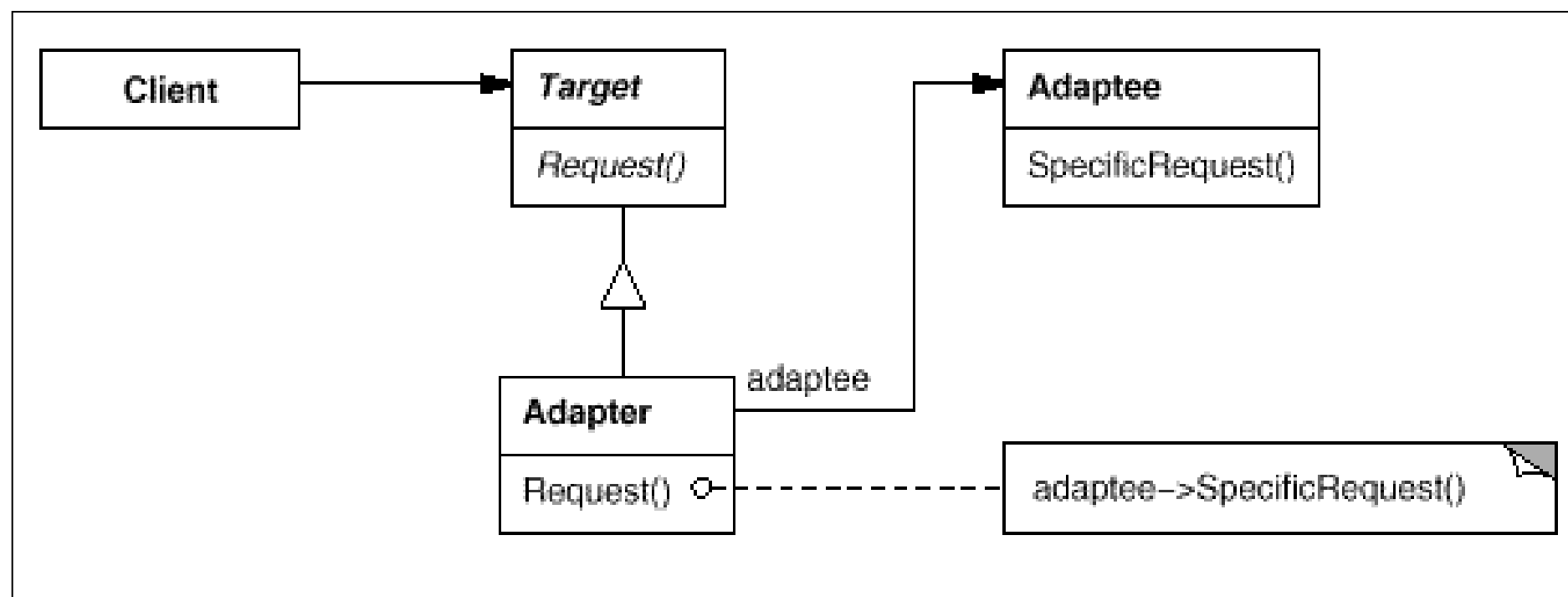
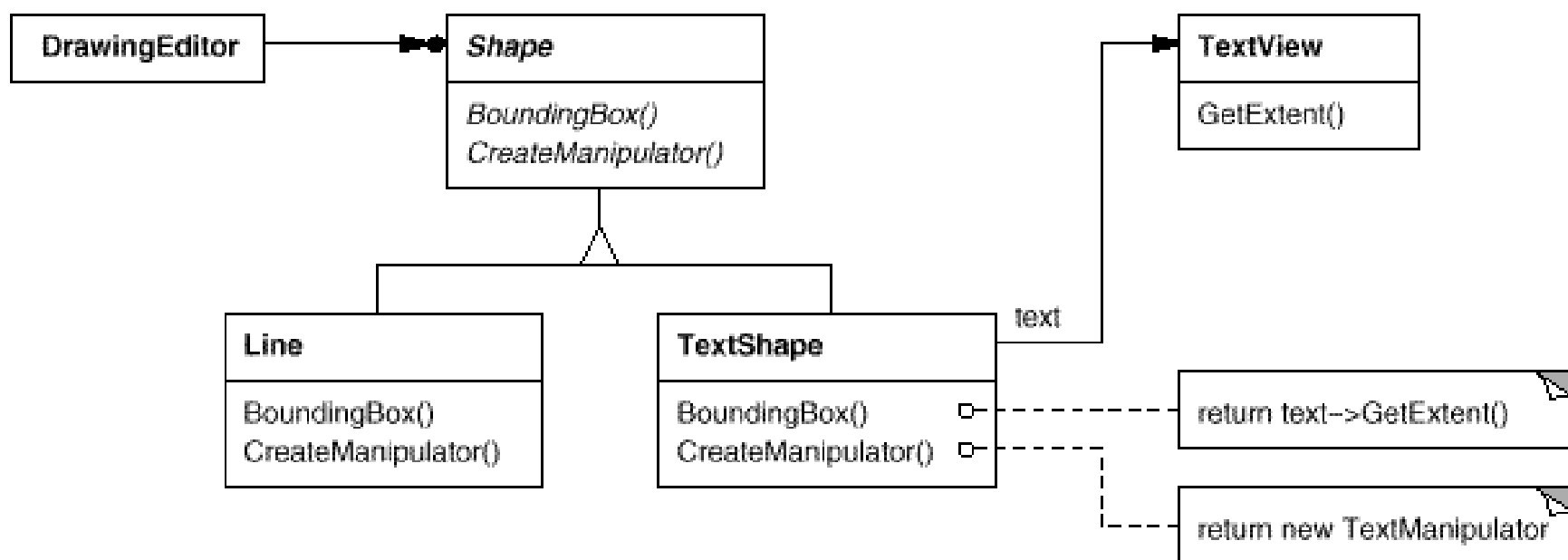
# Facade pattern



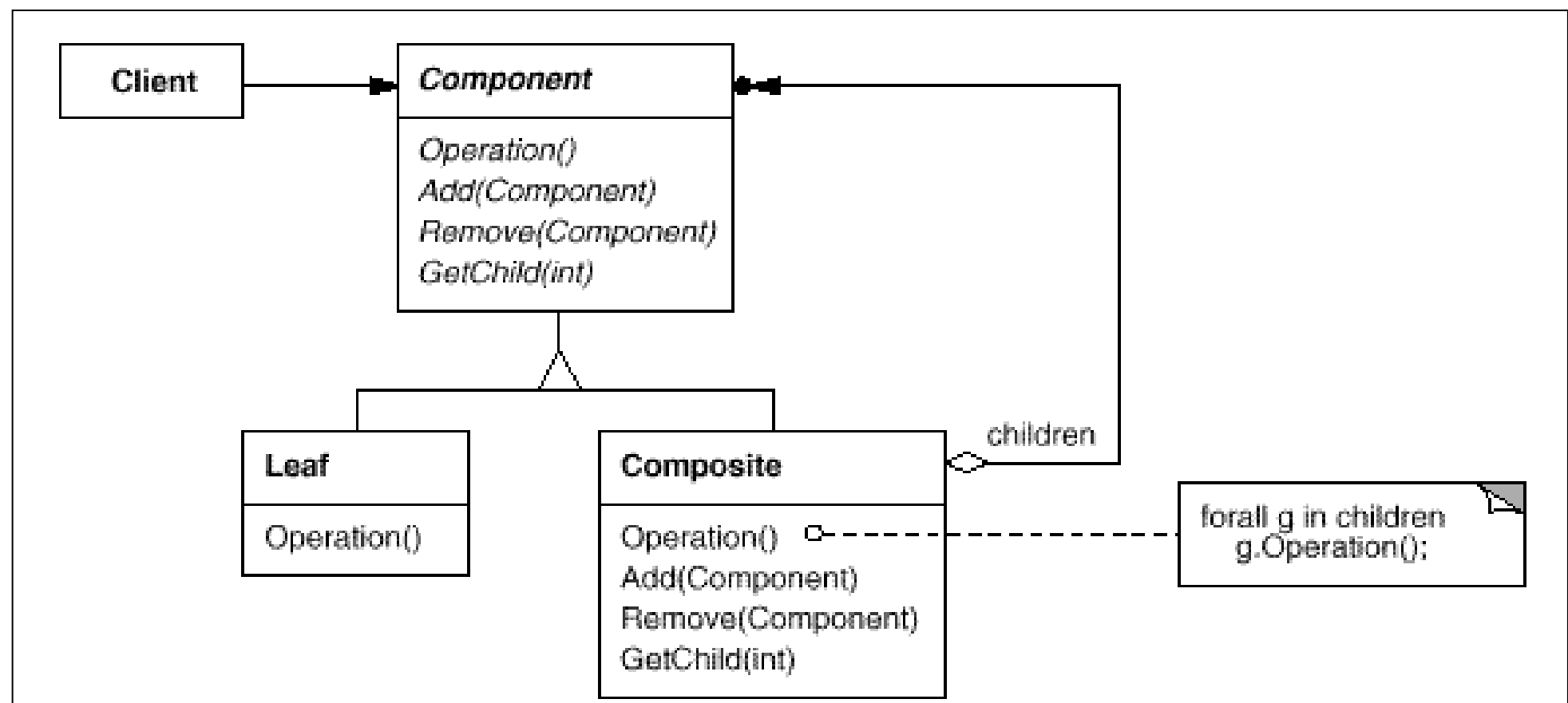
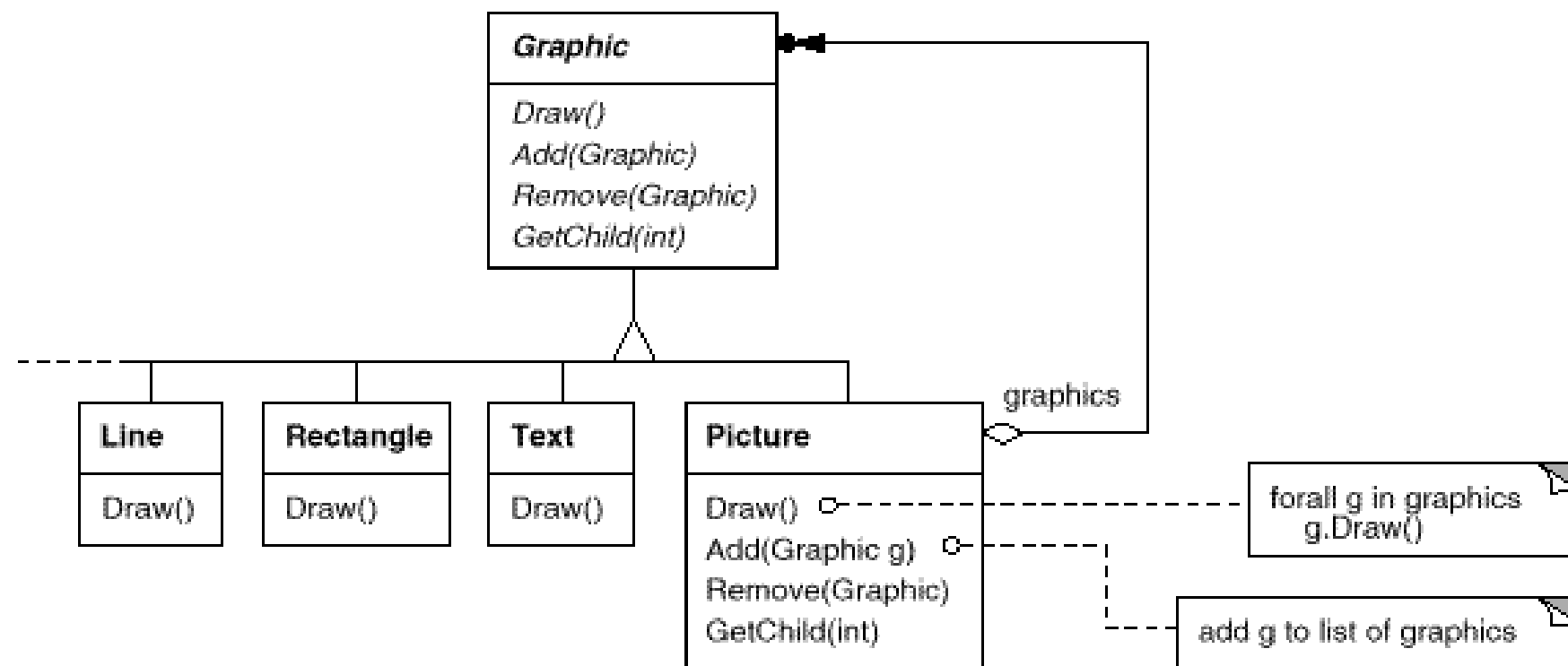
# Proxy pattern



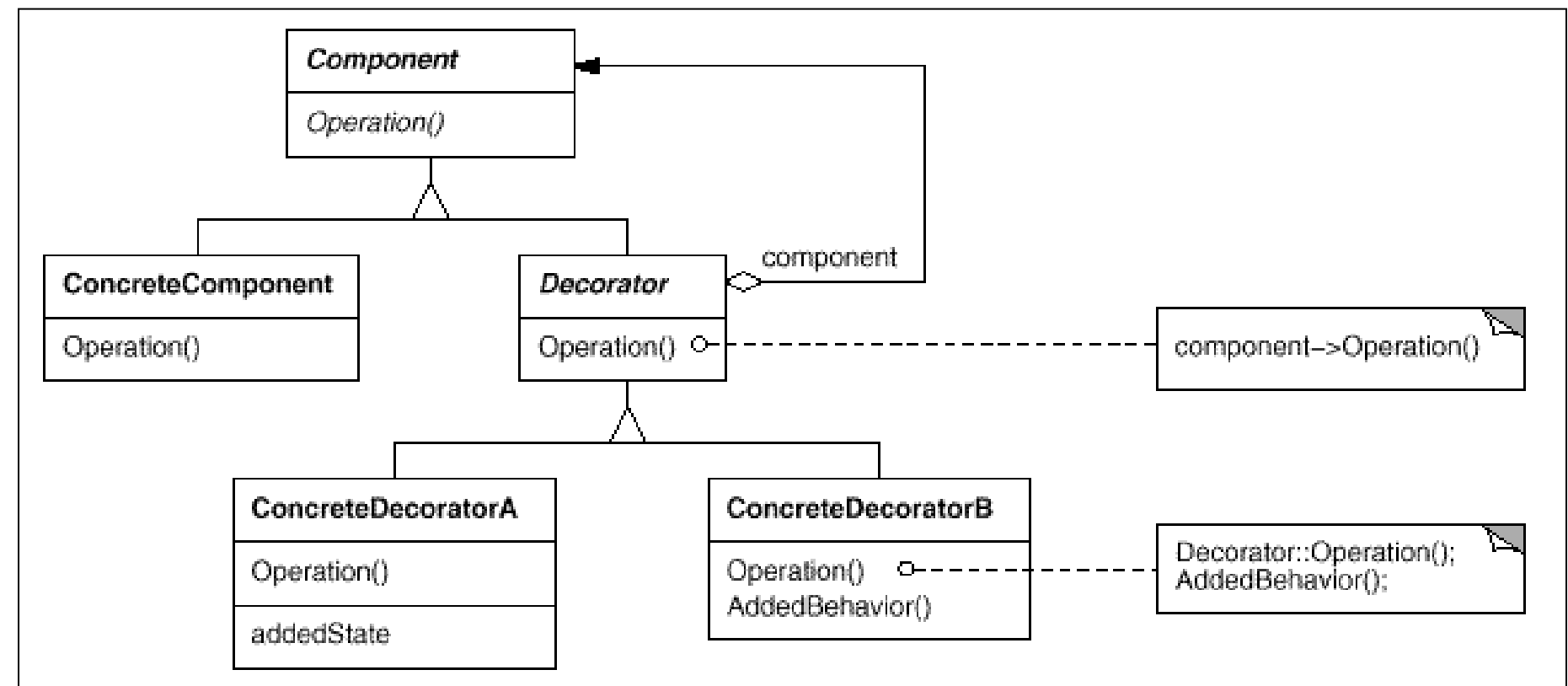
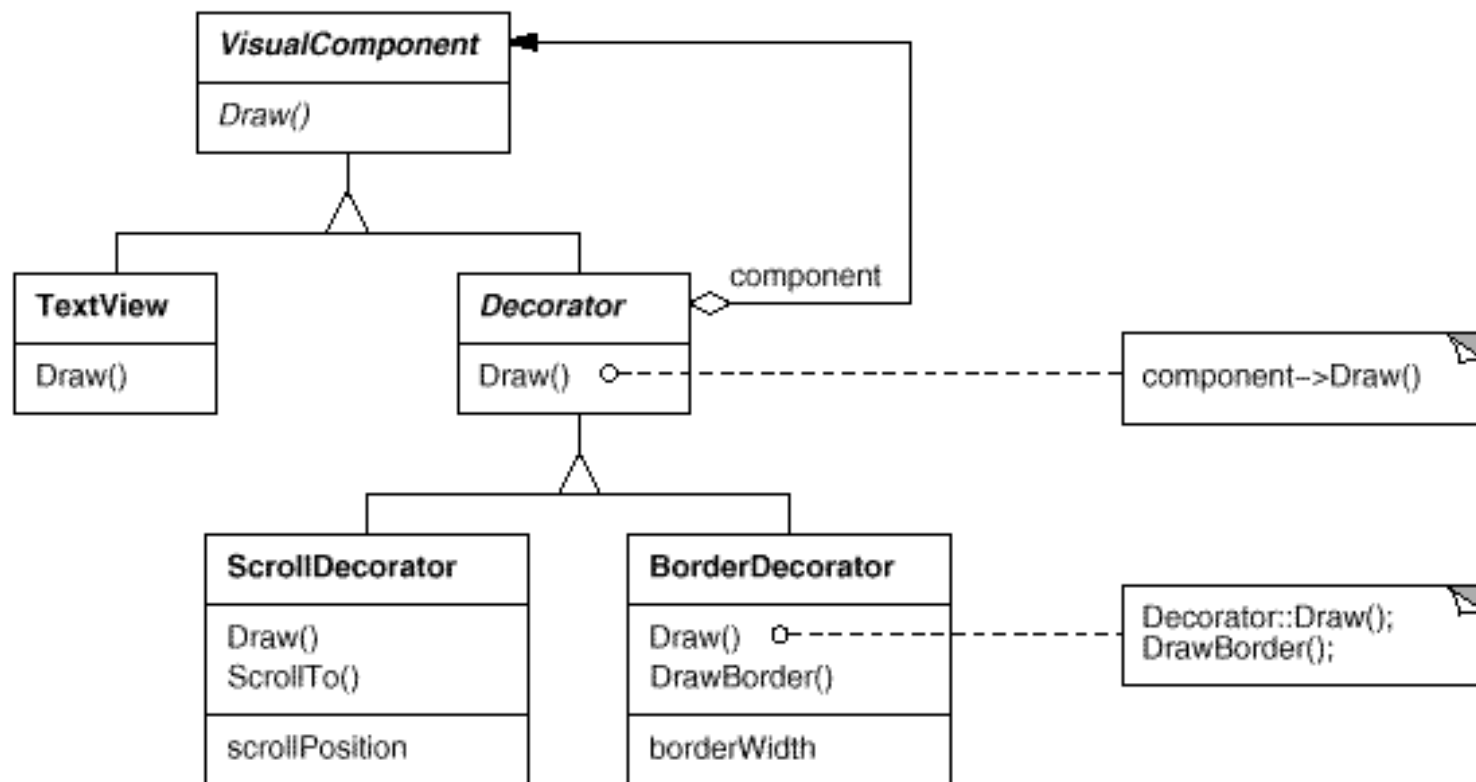
# Adapter pattern



# Composite pattern



# Decorator pattern





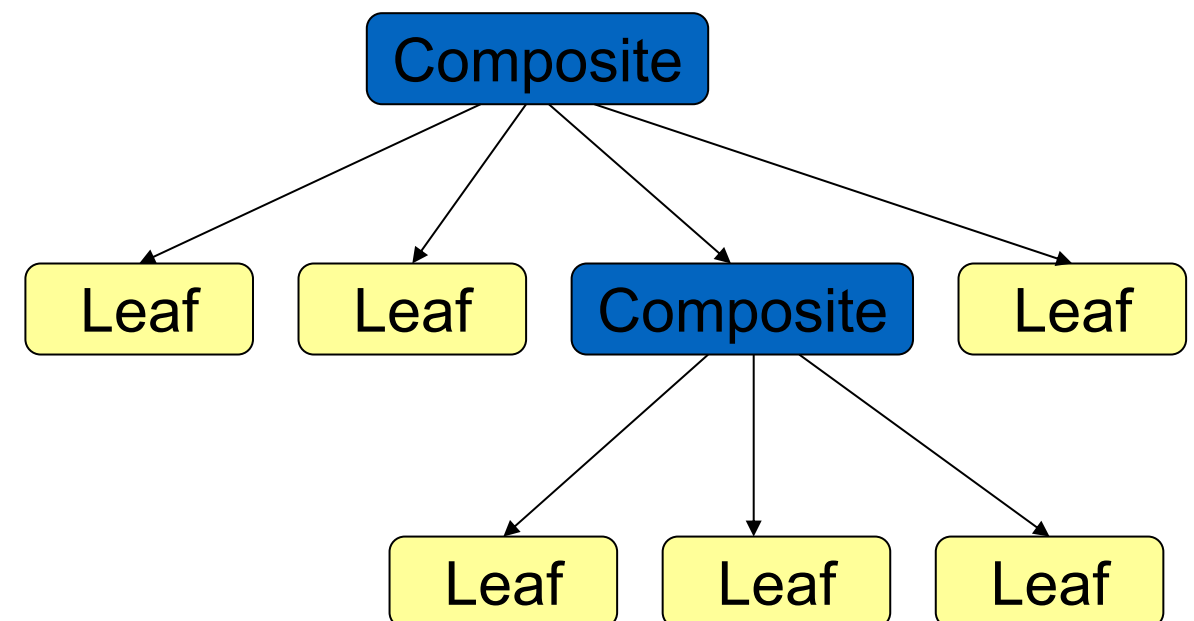
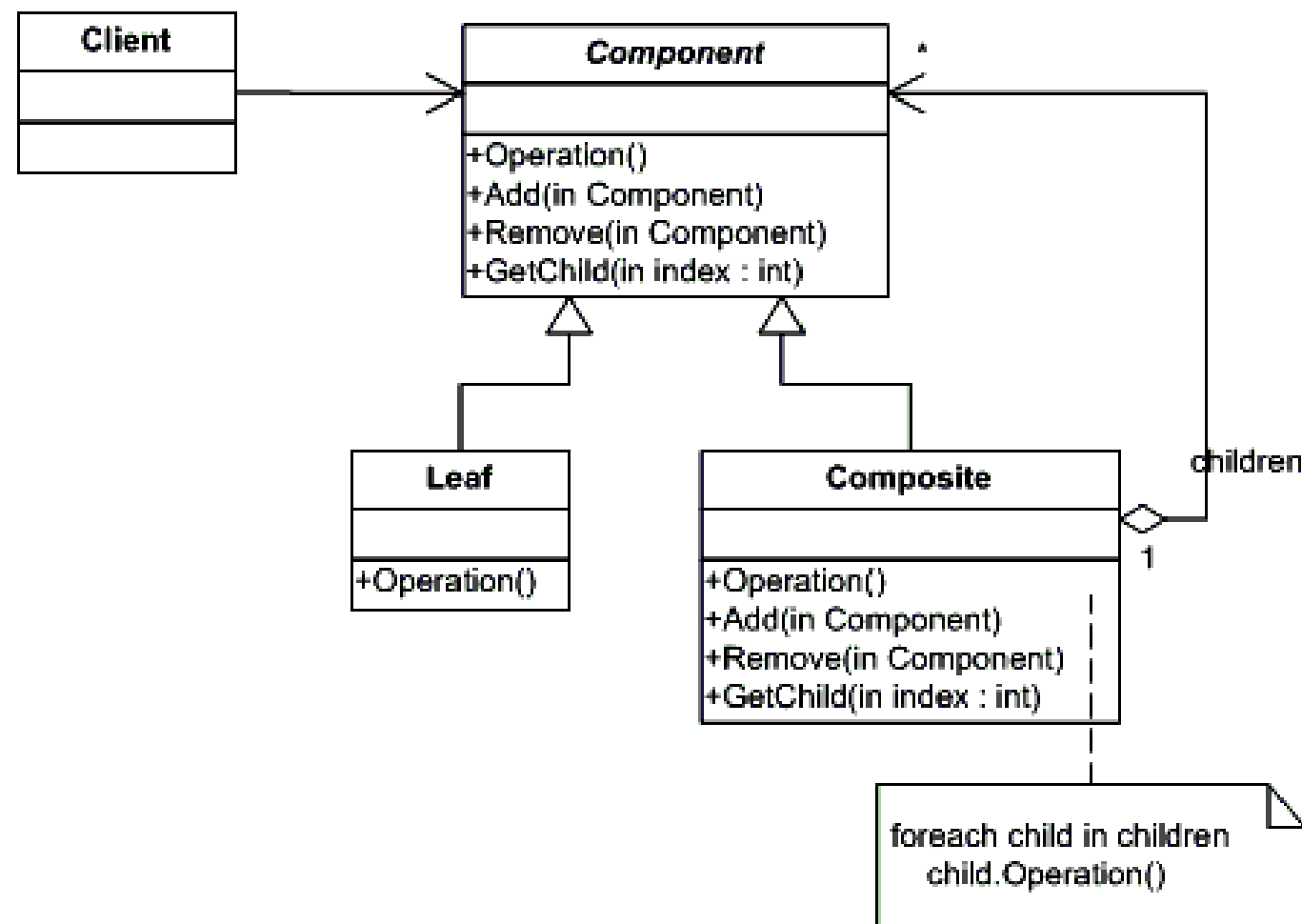
Ví dụ minh họa



# Ví dụ: Composite pattern

Composite pattern dùng để biểu diễn đối tượng theo cấu trúc dạng cây dựa trên sự phân cấp của các đối tượng.

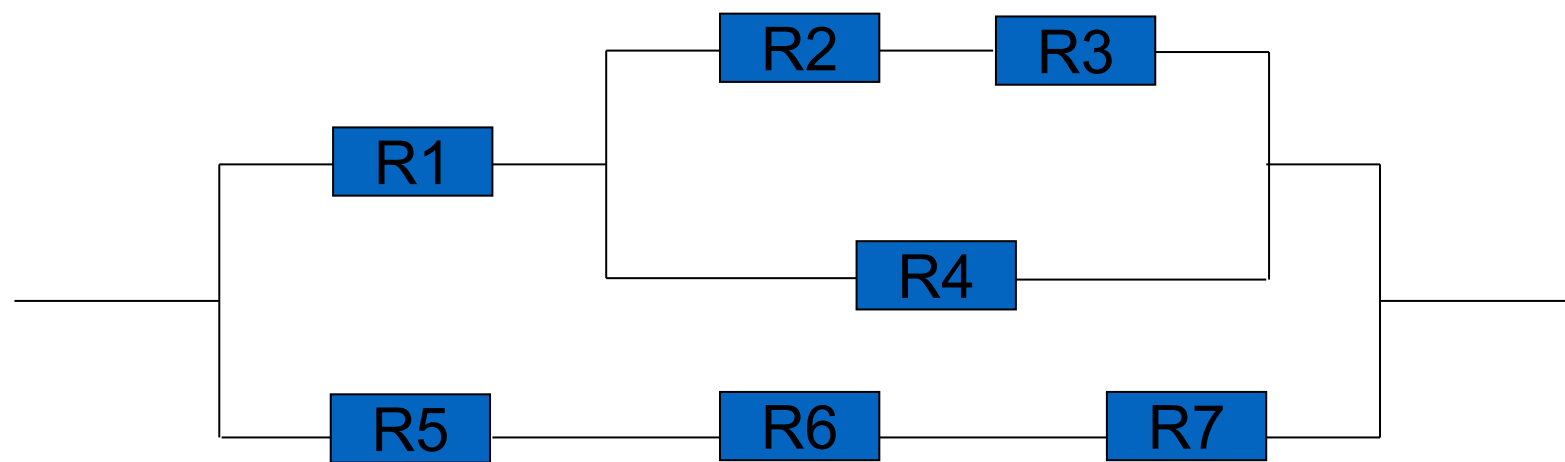
Composite pattern cho phép các client xử lý các đối tượng một cách độc lập và tổng hợp các đối tượng theo một dạng giống nhau.



Compose objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly.

## Yêu cầu bài toán

- Phân tích thiết kế hướng đối tượng để giải bài toán tính tổng trở cho mạch điện hỗn hợp gồm các điện trở được mắc trong mạch song song và nối tiếp.



$$R1 = 10 \, \Omega$$

$$R2 = 20 \, \Omega$$

.....

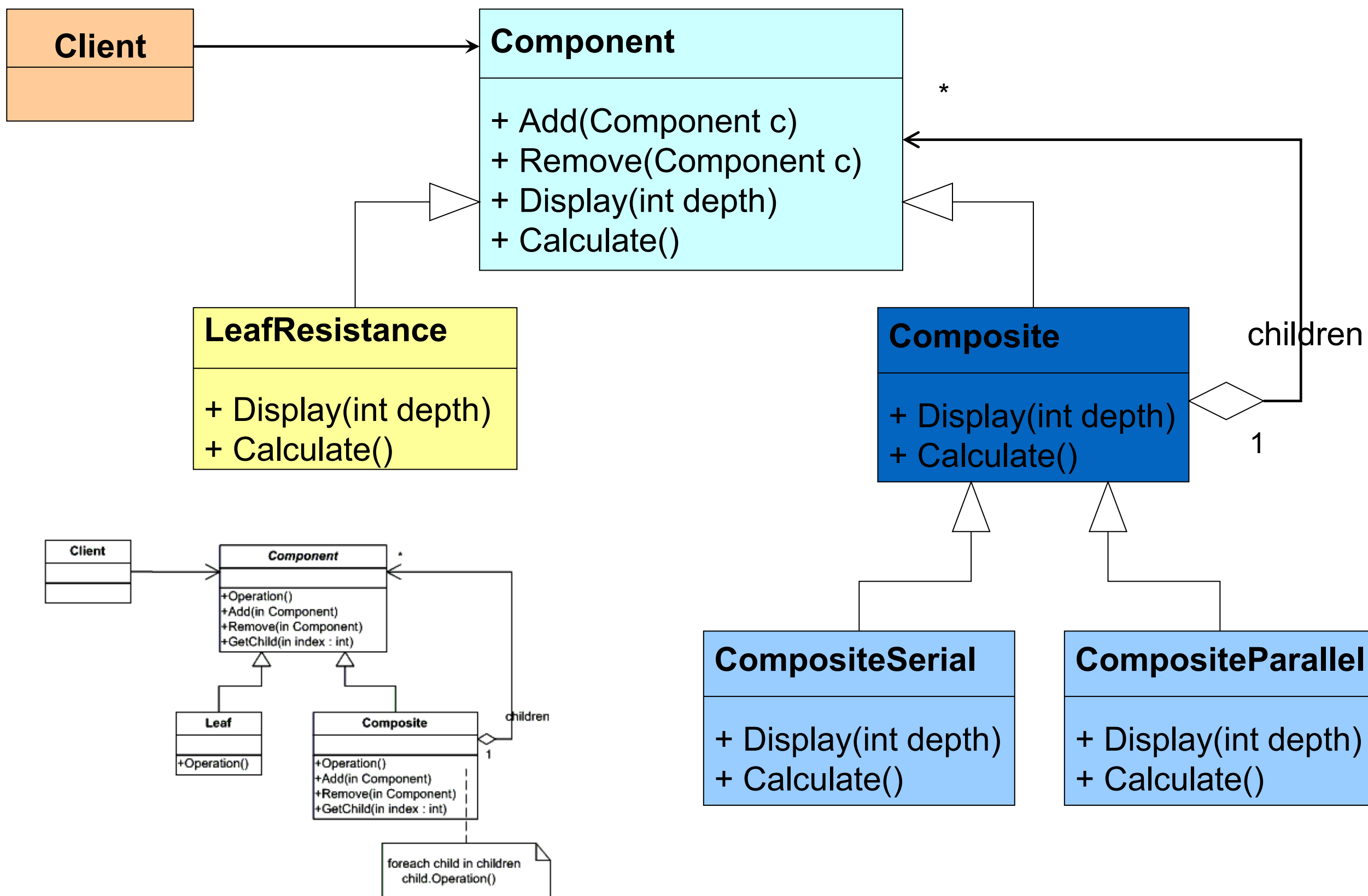
$$R7 = 70 \, \Omega$$

$$\text{Tổng trở} = ?$$

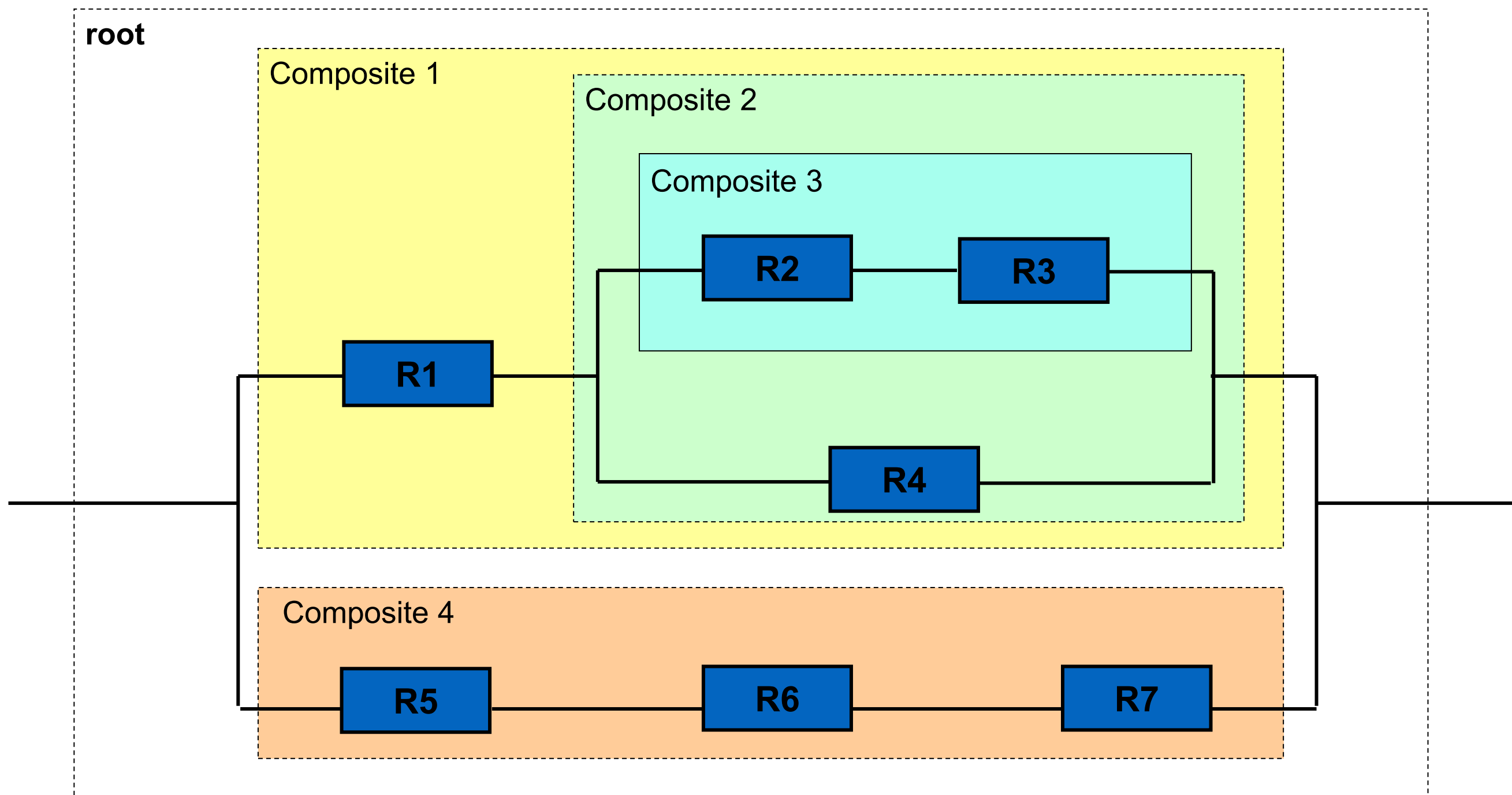
Các đối tượng trong mạch điện trên: (gồm có 12 đối tượng)

- Điện trở: 7 đối tượng
- Mạch nối tiếp: 3 đối tượng
- Mạch song song: 2 đối tượng

# Áp dụng Composite Pattern



# Tạo đối tượng



# Tạo đối tượng

## // Tạo cấu trúc mạch điện

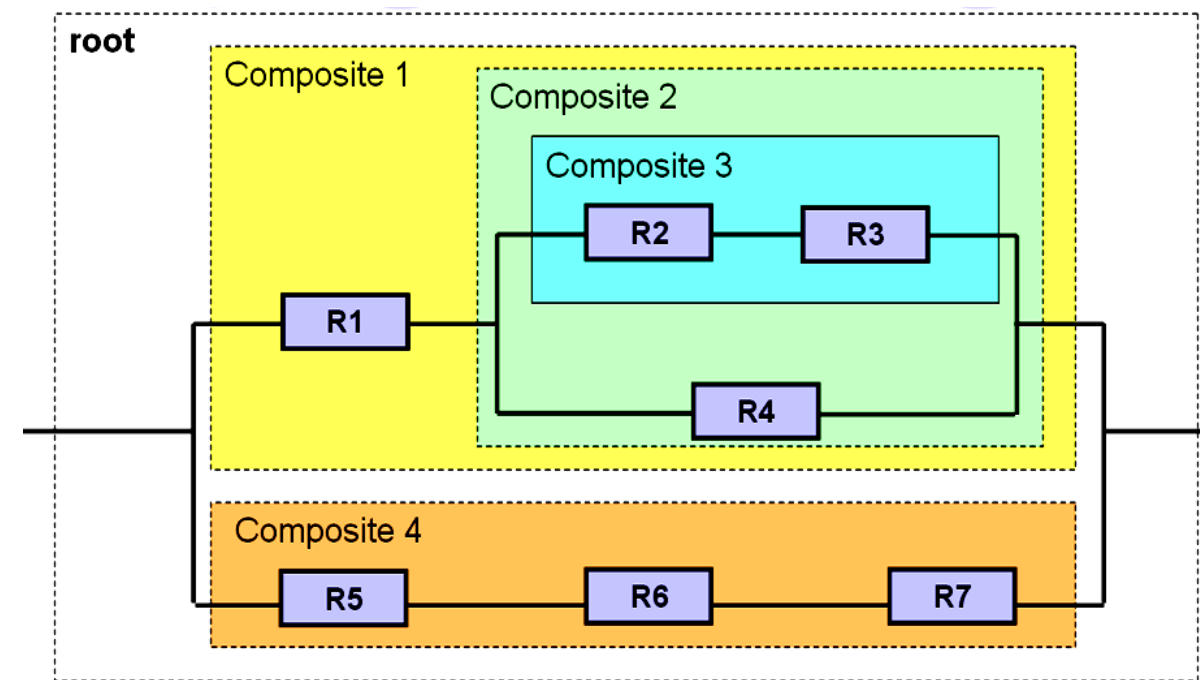
```
Composite root = new CompositeParallel("Root");
Composite c1 = new CompositeSerial("Composite 1");
Composite c2 = new CompositeParallel("Composite 2");
Composite c3 = new CompositeSerial("Composite 3");
Composite c4 = new CompositeSerial("Composite 4");
```

## // Gắn các mạch điện lại với nhau

```
root.Add(c1);
root.Add(c4);
c1.Add(c2);
c2.Add(c3);
```

## // Gắn các điện trở vào mạch điện

```
c1.Add(new LeafResistance("R1", 10));
c3.Add(new LeafResistance("R2", 20));
c3.Add(new LeafResistance("R3", 30));
c2.Add(new LeafResistance("R4", 40));
c4.Add(new LeafResistance("R5", 50));
c4.Add(new LeafResistance("R6", 60));
c4.Add(new LeafResistance("R7", 70));
```



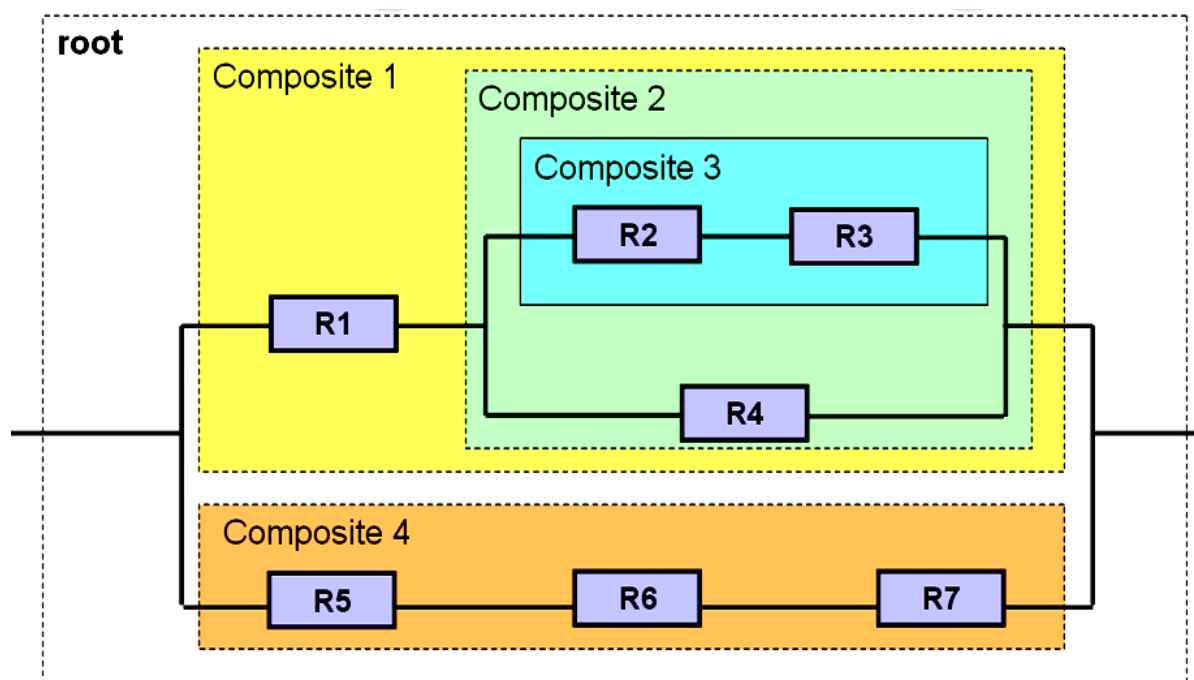
# Kết quả thực hiện chương trình

// Xuất mạch điện (dạng cây)

```
root.Display(0);
```

// Tổng trở mạch điện

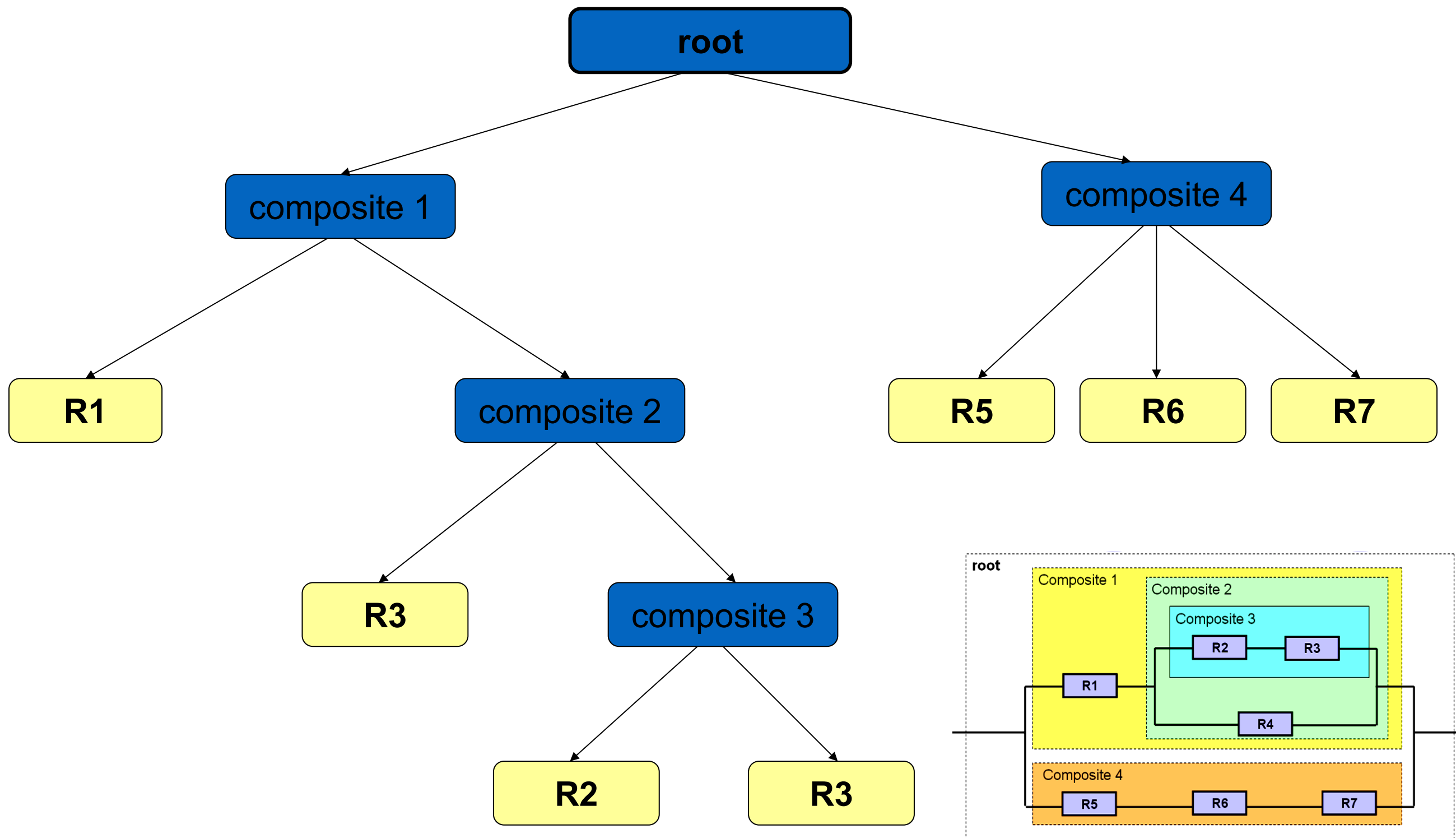
```
Console.Out.WriteLine("R = " +  
    root.Calculate().ToString());
```



```
[Root]
--[Composite 1]
----[Composite 2]
-----[Composite 3]
-----R2
-----R3
-----R4
----R1
--[Composite 4]
----R5
----R6
----R7
```

R = 27.3298429319372

# Cách thực thi hàm Calculate()





## Ví dụ: Storage Explorer

### **Viết chương trình thống kê:**

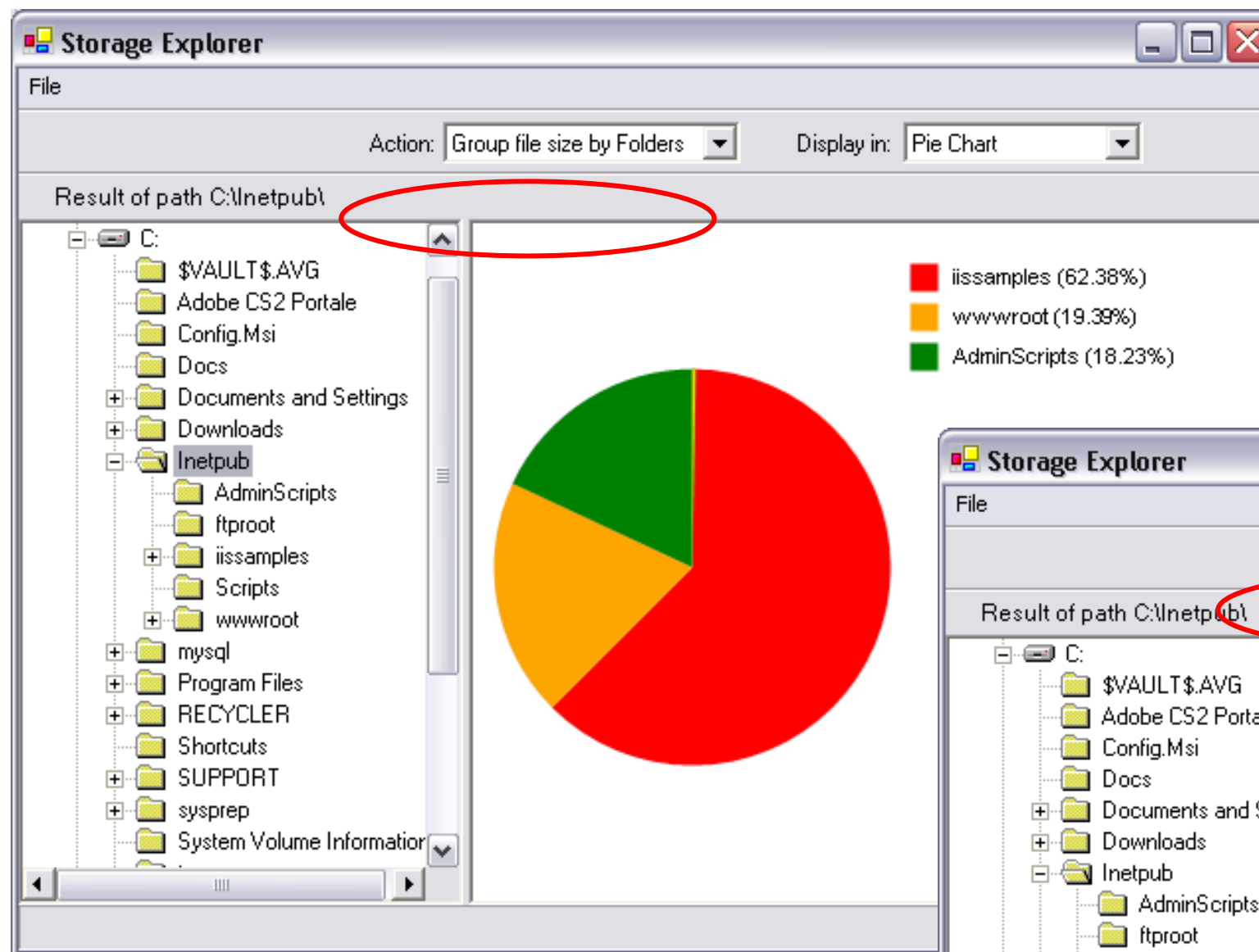
- Phần trăm dung lượng của các thư mục con bên trong một thư mục cha (nhóm theo thư mục).
- Phần trăm của các loại file bên trong một thư mục cha (nhóm theo loại file).

### **Và xuất ra kết quả (phần trăm) theo dạng:**

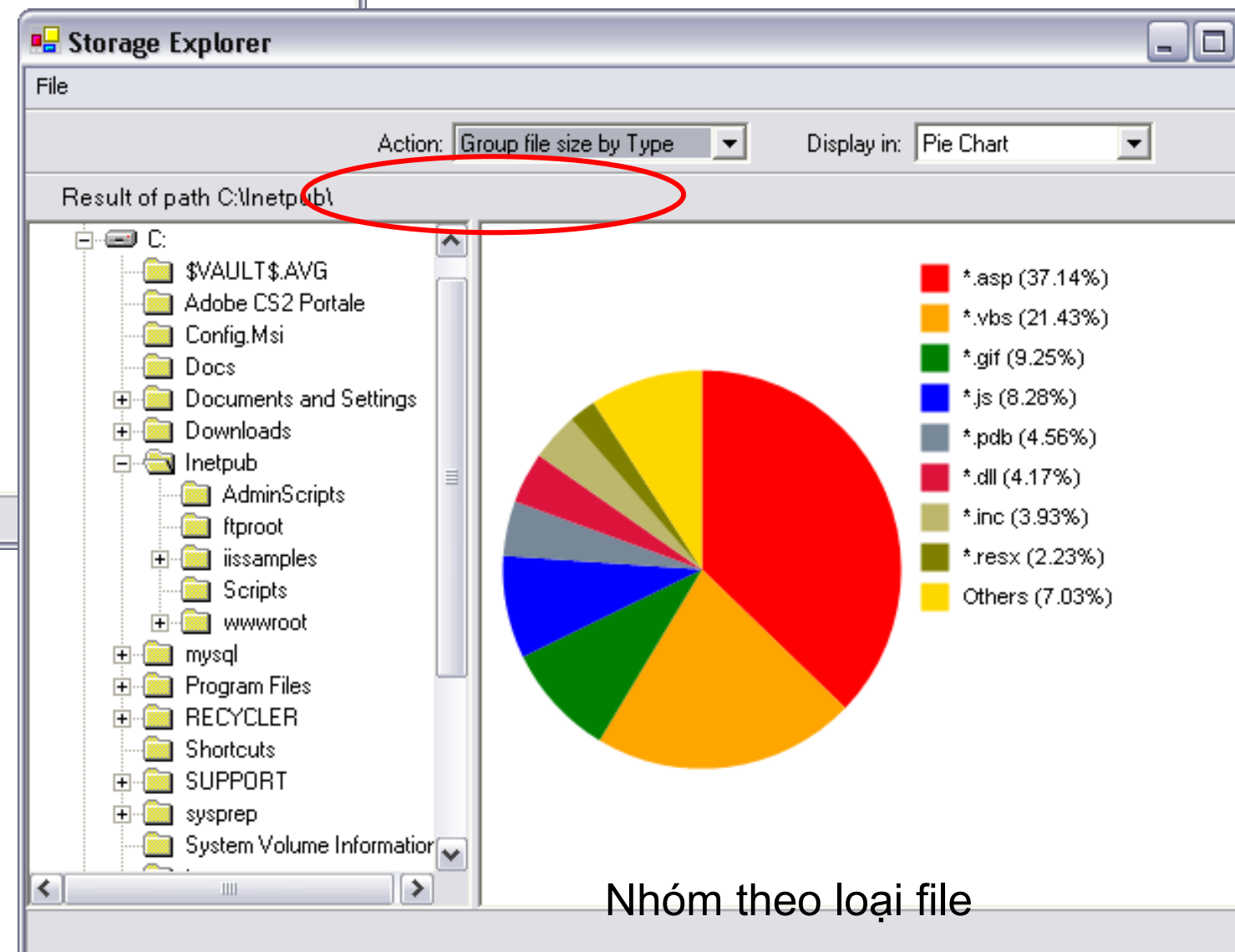
- Đồ thị dạng Pie.
- Đồ thị dạng Bar.
- Danh sách (ListView).



# Giao diện chương trình

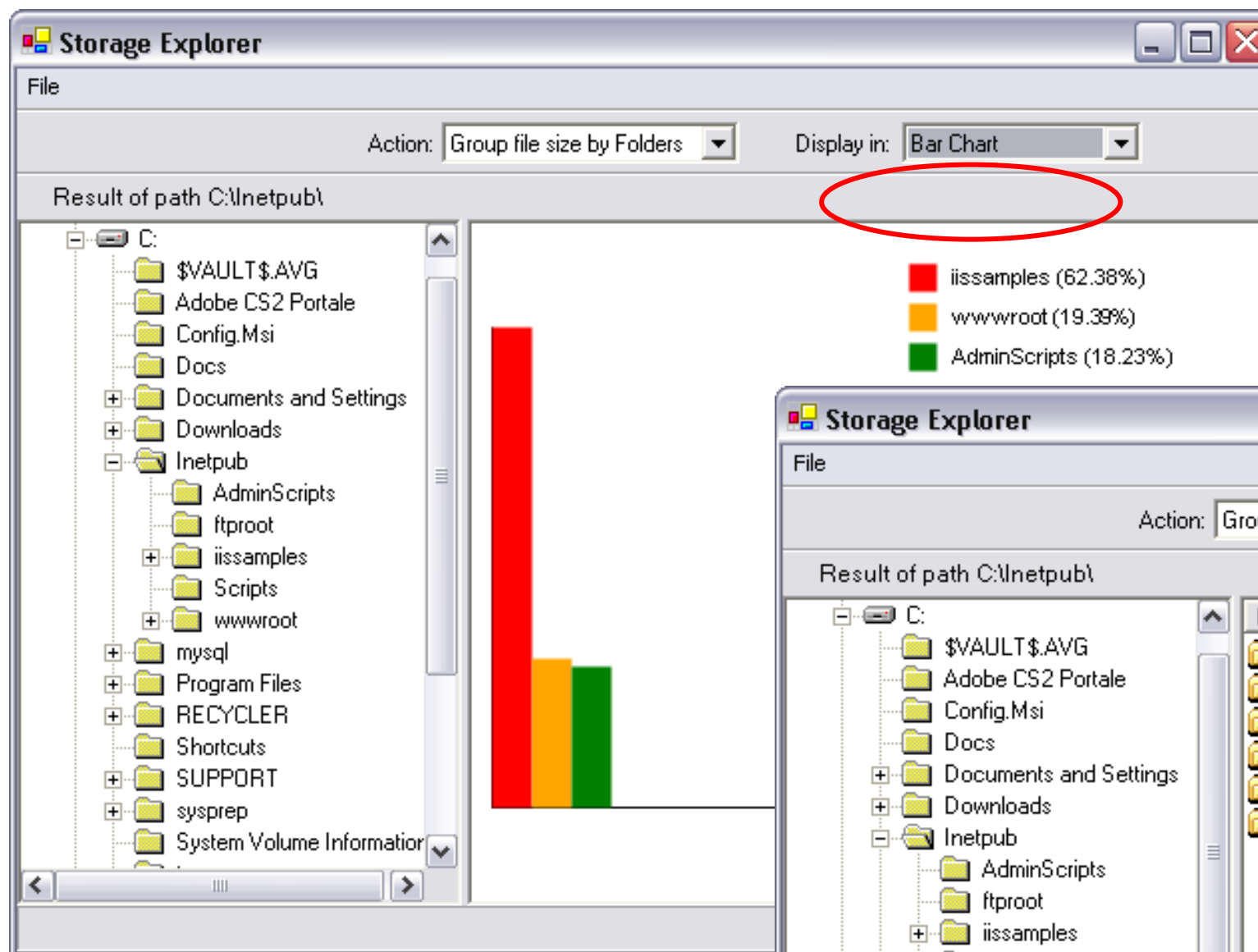


Nhóm theo thư mục



Nhóm theo loại file

# Giao diện chương trình



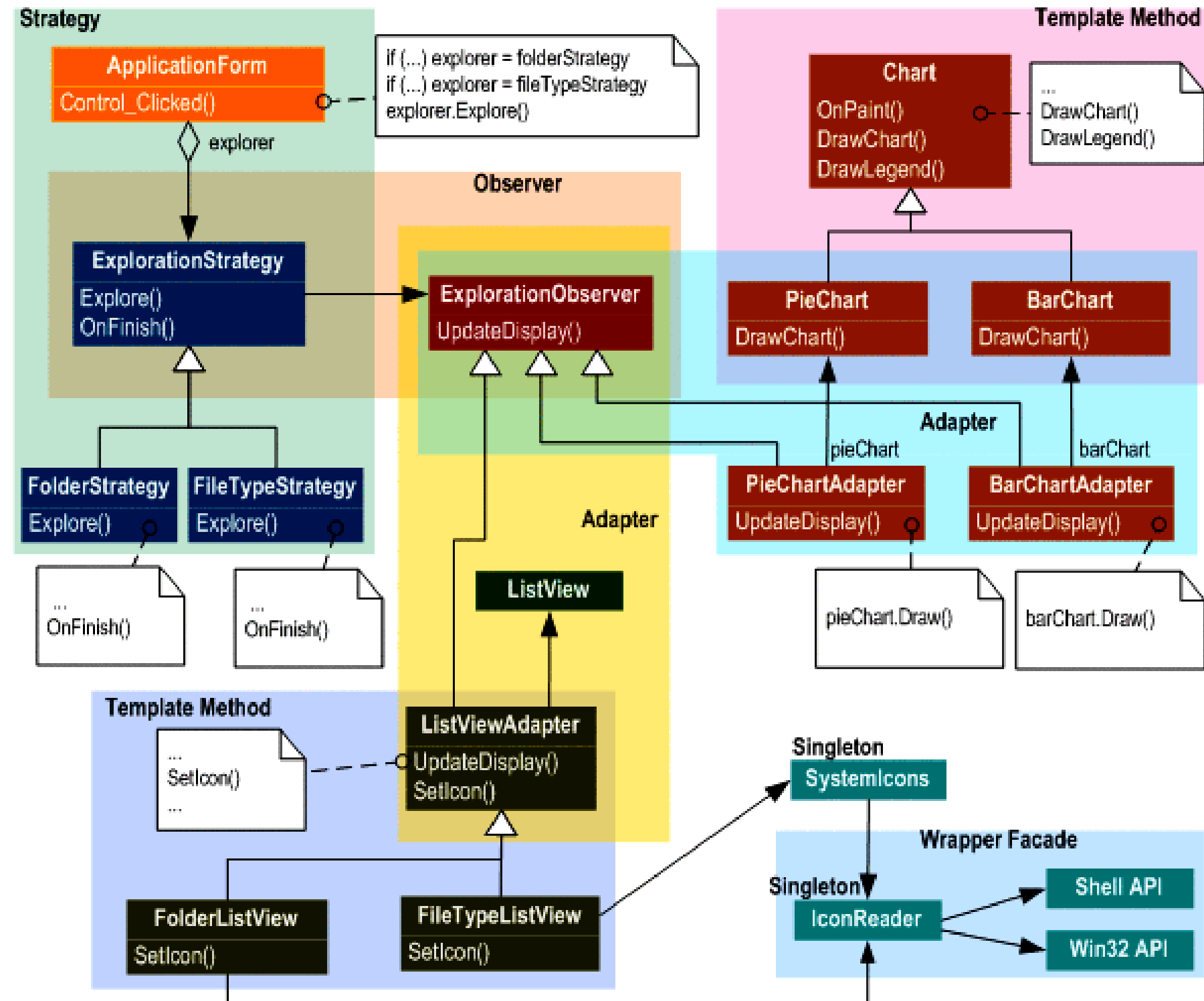
Xuất theo dạng Bar Chart

Storage Explorer window showing the result of path C:\inetpub\ in List View. The table displays the same data as the bar chart. The 'Display in:' dropdown is set to 'List View' and is circled in red.

| Name                | Size   | Percentage |
|---------------------|--------|------------|
| AdminScripts        | 159 KB | 18.23 %    |
| iissamples          | 546 KB | 62.38 %    |
| wwwroot             | 169 KB | 19.39 %    |
| ftproot             | 0 KB   | 0.00 %     |
| (Current Directory) | 0 KB   | 0.00 %     |
| Scripts             | 0 KB   | 0.00 %     |

Xuất theo dạng ListView

# Sơ đồ thiết kế ứng dụng



**Strategy Pattern:** Định nghĩa một họ các thuật toán và có khả năng lựa chọn giữa các thuật toán này.

**Observer Pattern:** Định nghĩa phụ thuộc 1 nhiều giữa các đối tượng, khi một đối tượng bị thay đổi trạng thái, nó sẽ báo động (notify) cho các đối tượng khác thay đổi trạng thái theo.

**Adapter Pattern:** Chuyển interface của một class bất kỳ thành interface của class mà client yêu cầu.

**Template Method Pattern:** Định nghĩa khung xương của một thuật toán để thực hiện một công việc nào đó, và một số bước của thuật toán sẽ được lớp con định nghĩa.

**Singleton Pattern:** Tại một thời điểm, chỉ cho phép tạo một Instance duy nhất của class.

**Wrapper Façade Pattern:** Gói các hàm và dữ liệu của API (theo hướng thủ tục) và tạo một interface của class (theo hướng đối tượng).

# Tài liệu tham khảo

- Sách điện tử (Ebook):
  - **Design Patterns Elements of Reusable Object Oriented Software**
  - **Design Patterns** – Minh Khai Pub
- Bài báo (Paper):
  - **Design Principles and Design Patterns** - Robert C. Martin
  - **Các nguyên lý lập trình hướng đối tượng** – Nguyễn Minh Huy  
*Tạp san khoa học cán bộ trẻ CNTT - DHKHTN*
- Internet:
  - **Huston Design Patterns**  
<http://home.earthlink.net/~huston2/dp/patterns.html>
  - **Data & Object Factory**  
<http://www.dofactory.com/Patterns/Patterns.aspx>

**CÁM ƠN ĐÃ LẮNG  
NGHE!**